

When Can Agents Safely Checkpoint, Fork, Restore, and Merge?

Exact Checking for Execution Edits

Abstract—Agent runtimes can Checkpoint an execution, Fork it, Restore a checkpoint, or Merge branches without restarting a task. We call these operations *execution edits* because they change which parts of the workflow will run next. An edit changes future work without undoing a tool action that has already been authorized or released. An unsafe edit can therefore authorize the same action twice, discard a result the workflow still requires, or conflict with a call that began before the edit. Existing Agent systems support Fork, Restore, or Merge, but neither these systems nor existing synthesis methods derive a requested edit’s safety conditions from the current execution record. We give an exact procedure for deciding whether an execution edit is safe. It either computes every safe way for the edited workflow to continue or proves that every possible implementation is unsafe. Given a registered workflow and execution record, a trusted controller derives the edited workflow and the outcomes that it must preserve. The procedure constructs the complete executions allowed by the workflow and policy, then removes any execution whose prefix would leave a still-required outcome with no safe completion. If no execution remains, it returns a finite, independently checkable proof that the edit has no safe implementation. Otherwise, the remaining executions contain every safe continuation, and the runtime permits exactly their prefixes. The main theorem covers Checkpoint, six execution edit forms, extensions, atomic enforcement, and the information every exact checker needs. Lean proofs and executable tests validate the theory and checker.

I. INTRODUCTION

Tool-using Agent runtimes can change an execution while it is in progress. They let an Agent Checkpoint an execution, Fork it into alternatives, Restore an earlier checkpoint, and Merge branches [1], [2], [3], [4], [5], [6]. These operations support exploration, recovery, and reuse without restarting the task. We call them *execution edits* because they change which parts of the workflow will run next.

Consider an Agent buying one laptop for a school. Before asking the school to approve the purchase, it saves checkpoint k . The school approves one purchase, and the Agent later chooses supplier R and authorizes payment. The Agent then restores k to compare supplier L and merges the L branch into the current execution. If the runtime sees only the restored workspace, it may request the same approval again or continue with L even though payment to R is already authorized [7]. Either result violates the intended workflow because the first repeats a one-time approval and the second combines incompatible supplier actions.

Detecting this failure requires more than the restored checkpoint. The execution record contains four facts that the checker needs: (1) the workflow and its required outcomes, (2)

which calls refer to the same action, (3) earlier authorizations and call progress, and (4) which rules govern a call that overlaps the edit. Each field can change the correct answer. Equal workspaces, an execution trace alone, an authorization log alone, or a replacement workflow proposed by the Agent cannot reconstruct this record.

Existing approaches cover parts of this problem but do not derive its complete safety condition. Agent systems provide branching, staged effects, transactions, or recovery checks [8], [9], [10], [11], [12], [13], [14], [15]. Control and synthesis choose behavior after a model of possible executions and desired final states has been supplied [16], [17], [18], [19]. Neither line of work derives from a recorded execution everything that a particular edit must preserve before it takes effect.

Our key idea is that an execution edit must preserve both past actions and still-required outcomes. Every outcome required before the edit must remain required, already be satisfied by the recorded execution, or be explicitly removed by the same authority that required it. The Agent identifies a registered edit. The platform authenticates the registered workflow, and the trusted controller derives the edited workflow and inherited requirements from that record rather than from Agent text. The Agent therefore cannot obtain a favorable answer by submitting a replacement workflow that hides inconvenient facts.

This idea creates three technical problems. First, calls copied by Fork or Restore may refer to the same underlying action, so the controller must distinguish a new authorization from reuse of an earlier authorization and return value. Second, after every allowed partial execution, every outcome that is still required and still possible must retain a safe way to finish. The checker must enforce this condition without forbidding additional safe behavior. Third, the answer must take effect atomically with calls that overlap the edit and remain valid if the runtime or machine stops unexpectedly and later recovers.

We give an exact checker that either computes every safe way for an edited workflow to continue or proves that every possible implementation is unsafe. For a requested edit, the controller uses the registered workflow and current execution record to construct every allowed complete execution. It removes an execution whenever one of its partial executions would leave a still-required outcome with no safe completion. If no execution remains, the controller returns an independently checkable proof that no safe implementation exists. Otherwise, the remaining executions contain every safe way for the edit

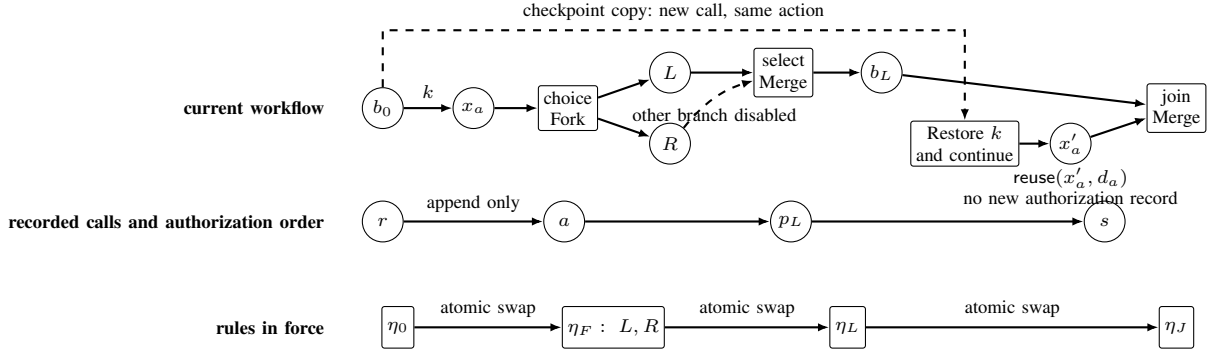


Fig. 1. Fork, Restore, and Merge change the current workflow but do not erase recorded calls. Restore creates a new workflow call that refers to the same action as the original call. The atomic rule update orders both creation and reuse of an authorization record against the rule change.

to proceed. Before the edit takes effect, the runtime checks the current execution record again and atomically switches to the newly computed rules. Every overlapping call is therefore checked entirely under either the old rules or the new rules. Figure 1 illustrates this process for an approval workflow containing Fork, Merge, Restore, and calls that overlap the atomic rule update. Section II derives the example step by step. After Restore, the repeated approval call reuses the earlier authorization record instead of authorizing the approval again.

a) Contributions:

- 1) **Safety model.** We define safety for Agent Checkpoint, Fork, Restore, and Merge from the execution record rather than from a replacement workflow proposed by the Agent. The model preserves past actions and every still-required outcome, and it proves that every exact checker needs the four facts identified above.
- 2) **Exact checking and runtime implementation.** We construct an executable checker that returns either every safe way for the edit to proceed or a proof that none exists. Checkpoint has an exact local check, and each of the six registered execution forms is accepted exactly when a safe runtime implementation exists. The runtime puts the result into effect atomically, preserves safety across repeated edits and restarts, and has the same Agent-visible behavior as an ideal atomic machine.
- 3) **Mechanization and executable validation.** Six Lean modules mechanize registration, all six edit rules, the atomic rule update, calls concurrent with that update, and preservation across finite executions. Nineteen paper-specific tests and measurements over 2–128 outcomes exercise the executable checker.

II. EXECUTION-EDIT SAFETY

Imagine an Agent buying one laptop for a school. It reserves the budget, saves checkpoint k , receives one purchase approval, and then compares suppliers L and R . A careless Restore or Merge could request approval twice, pay both suppliers, or permit shipment before the selected supplier is paid. A safe execution approves the purchase once, pays exactly one supplier, and permits shipment only after that payment.

We formalize this story as an approval workflow that authorizes at most one order. The Agent reserves the budget and saves checkpoint k before approval. The approval is then authorized once, creating an authorization record that Restore does not delete. The Agent next requests a Fork between suppliers L and R . The workflow allows two outcomes: order from L or order from R . In either outcome, payment authorization must precede shipment authorization:

$$\begin{aligned} o_L : x_L^p < x_L^s, \\ o_R : x_R^p < x_R^s, \end{aligned} \quad (1)$$

where x_i^p and x_i^s are payment and shipment calls. A *workflow call* x is one place where the workflow can invoke a protected tool. The function $q_{\text{act}}(x) = d$ records which external action the call refers to. Calls that refer to the same action share one authorization record.

In this example, both shipment calls authorize the same one-order shipment:

$$q_{\text{act}}(x_L^s) = q_{\text{act}}(x_R^s) = d_s.$$

The two payment calls instead refer to different actions d_L^p and d_R^p .

The authorization log Δ records newly authorized actions in order. The notation $\text{lab}(\Delta)$ keeps only their authorization labels. Budget reservation contributes r , approval contributes a , the two payments contribute p_L and p_R , and shipment contributes s . At the Fork request, $\text{lab}(\Delta) = ra$. The policy allows prefixes of rap_Ls and rap_Rs , but never both payments or a payment before approval. Although the supplier outcomes are exclusive, both must still have a safe way to finish when the Fork is checked.

A. Accepted and Rejected Forks

The Agent sends $u = \text{ForkChoice}(b, \sigma)$, which names a current branch and a registered edit rule. The trusted controller constructs the edited workflow from the execution record. It also derives the outcomes that remain required and the action referred to by every workflow call.

Write M_o for the ways to complete outcome o while obeying the policy. Each outcome has one allowed call order:

$$\begin{aligned} z_L &= \text{new}(x_L^p, d_L^p) \text{new}(x_L^s, d_s), \\ z_R &= \text{new}(x_R^p, d_R^p) \text{new}(x_R^s, d_s). \end{aligned}$$

Thus $M_{o_L} = \{z_L\}$ and $M_{o_R} = \{z_R\}$. These sequences contain the calls after the edit, while the existing log contributes the earlier prefix ra . Each supplier outcome is preserved, and every permitted prefix can still finish safely. The verdict is therefore *Accept* because both supplier outcomes remain possible, every permitted prefix is policy safe, and each such prefix can still reach a complete outcome. If policy excludes rap_R s, then $M_{o_R} = \emptyset$, and the Fork is rejected because pruning supplier R would change its meaning. Rejection returns a proof listing the order in which completions were removed and the reason for each removal, rather than letting the Agent weaken the requested edit. The verdict in this variant is *Reject* because checking only the surviving L branch would silently weaken the requested Fork.

B. Why Safe Endpoints Are Not Enough

The previous rejection is visible at the start because one outcome has no safe completion. A harder failure occurs when every outcome is initially safe in isolation, but no single set of runtime rules can keep all of them completable. Let X, Y, Z be single-call contracts with authorization labels x, y, z , respectively, and consider

$$X \otimes (Y \oplus Z) \quad \text{under} \quad \mathcal{A} = \text{Pref}\{yx, xz\}.$$

The two outcomes have safe complete traces yx and xz . However, executing the shared x call first keeps the $X \otimes Y$ outcome possible while leaving only the forbidden continuation xy . Allowing x is therefore unsafe, while forbidding x destroys the $X \otimes Z$ outcome. The exact verdict is *Reject*, even though an outcome-by-outcome check would accept. Section IV detects this failure by repeatedly removing executions that create such a bad prefix.

C. Calls During the Atomic Rule Update

Figure 1 separates the current workflow, the authorization log, and the rules that must change atomically.

a) *New authorization during the rule update:* After the shared approval and before either payment, the Agent requests $\text{MergeSelect}(g, L, \sigma)$. If the payment call x_R^p from the old supplier- R branch is authorized first, it selects R and changes $\text{lab}(\Delta)$ from ra to rap_R . The controller must therefore check the Merge again and reject it. If the Merge rule update occurs first, the old rules stop applying and every current call receives a new authorization token before any later x_R^p can be authorized. Leaving the old rules active would allow both payments.

b) *Authorization reuse during the rule update:* Now consider the Restore that keeps both branches in figure 1. A later Restore of k creates a new approval call x'_a , but both approval calls refer to the same action d_a . The restored call is new, while the underlying action remains d_a . Because d_a already has an authorization record, executing x'_a reuses that

record. The restored branch therefore advances past approval without creating another authorization record. A concurrent Merge or Restore result computed before that progress is now out of date even though the authorization log has not changed. The final check must therefore compare both the authorization log and the ordered record χ of executed workflow calls. For this RestoreLive request, the runtime puts the newly computed rules into effect only if the execution record is unchanged. Otherwise, it discards the outdated result and checks the request again against the current record.

Together, these executions give the paper's end-to-end safety rule. An edit satisfies this rule when (1) it preserves every outcome still required by the execution record, (2) every allowed step obeys policy and can still reach a complete execution, (3) every required outcome that remains possible after those steps still has a safe way to finish, and (4) the new rules and authorization tokens take effect atomically with concurrent calls. Section III states this rule precisely, and section IV constructs the exact checker.

III. MODEL AND SECURITY OBJECTIVE

The model leaves the Agent's internal reasoning unrestricted and places every protected tool call under trusted runtime control. Each call identifies its registered place in the workflow, presents an authorization token, and submits a concrete tool request. The runtime either creates a new authorization, reuses an authorization already recorded in the log, or rejects the call. Calls governed by one policy, one authorization log, and one atomic rule change form a *policy domain*. Policy domains with disjoint actions, logs, and policies compose independently. Every individual state is finite, while the theory covers executions of arbitrary finite length. The *execution record* stores the workflow and checkpoints, which calls refer to the same action, earlier authorizations and call progress, queued requests, and the rules currently in force. The trusted runtime protects this record from Agent modification.

A. Workflow outcomes and call order

a) *Basic objects:* We capitalize *Agent* for the untrusted principal requesting execution edits. A *workflow call* x is one place where the workflow can invoke a protected tool. The function $q_{\text{act}}(x) = d$ records the external action referred to by that call. Several workflow calls may refer to the same action d , in which case they share one authorization record. The first authorized call creates that record, and calls copied by Fork or Restore later reuse it. The execution record also stores where every copied workflow region came from. T, Δ, χ denote the current workflow, authorization log, and ordered record of processed workflow calls.

A pomset $p = (X_p, \prec_p)$ is a finite set of workflow calls with a strict partial order. Its linearizations $\text{Lin}(p)$ are the words containing each member of X_p once and respecting $\prec_p$. A *workflow contract* $\mathcal{C} = (O, \{p_o\}_{o \in O})$ assigns one pomset to each member of a finite nonempty outcome set O . Each index o names one allowed completion outcome. Two outcomes remain distinct even if the calls still required

for them later become identical. Write $X_C = \bigcup_{o \in O} X_{p_o}$ and $\text{CL}(\mathcal{C}) = \bigcup_{o \in O} \text{Lin}(p_o)$. We require the runtime to distinguish completion from continuation.

$$v, w \in \text{CL}(\mathcal{C}) \wedge v \preceq_{\text{pref}} w \implies v = w.$$

This condition permits incomparable traces and different outcomes with the same trace, so it does not require deterministic outcomes. It requires a registered workflow call whenever continuing after a shared partial execution performs more work. The interface represents a choice between stopping and continuing by an authenticated terminal workflow call handled by the controller.

Contracts use the three partial constructors below. Each constructor requires operands with disjoint workflow calls and preserves the condition that no complete call sequence is a strict prefix of another.

$$\begin{aligned} O_{C \oplus \mathcal{D}} &= (\{L\} \times O_C) \uplus (\{R\} \times O_{\mathcal{D}}), \\ p_{(L,o)} &= p_o, \quad p_{(R,v)} = p_v, \\ O_{C \otimes \mathcal{D}} &= O_C \times O_{\mathcal{D}}, \\ p_{(o,v)} &= p_o \uplus p_v, \\ O_{C \triangleright \mathcal{D}} &= O_C \times O_{\mathcal{D}}, \\ p_{(o,v)} &= p_o \triangleright p_v. \end{aligned} \quad (2)$$

Before adding a registered or copied operand, the controller deterministically assigns fresh workflow-call IDs and extends ρ while retaining the authenticated actions. Well-formedness already makes carried operands disjoint. In equation (2), $\uplus$ adds no cross-order and $\triangleright$ orders every event of p before every event of q , while choice uses a tagged union of outcome indices and parallel and sequence use Cartesian products. Consequently, choice retains each arm's full indexed family, parallel pairs outcomes while permitting every causal linearization, and sequence adds a barrier. After call prefix w , $\mathcal{C}/w$ retains each outcome whose pomset has a linearization prefixed by w , then removes the workflow calls in w and their order edges. The expression is undefined if no outcome remains. This operational "remaining contract" preserves outcome and workflow call identity.

Lemma 1 (Completion remains unambiguous). *For every defined $\mathcal{C}/w$, no complete sequence is a strict prefix of another, and either every retained pomset is empty or none is empty.*

Proof. If a remaining suffix v strictly prefixes v' , then wv strictly prefixes wv' in $\text{CL}(\mathcal{C})$. If one retained pomset is empty and another is nonempty, choose a nonempty linearization v of the latter. Then w strictly prefixes wv . Both contradict the condition that no complete sequence strictly prefixes another. $\square$

B. Execution Record

1) *Recorded structure and current workflow:* The workflow part of the execution record is

$$H = (\omega, G, T, K, \Sigma_{\zeta}, \rho, \beta, \chi, \text{ver}, \eta). \quad (3)$$

ω identifies the policy domain, and η identifies the version of the rules currently in force. G records the finite sequence of workflow versions. Its node IDs, registered contracts, signed outcome requirements, and Fork, Restore, and Merge records do not change. Each branch node b carries an immutable registered contract Γ_b . Its ordered progress record χ_b lists the workflow calls already processed on that branch. The immutable records form a directed acyclic graph (DAG) whose edges record typed Fork, Restore, and Merge operations. T is the current workflow.

$$T ::= b:\mathcal{C} \mid T \oplus_g^s T \mid T \parallel_g T \mid \text{join}(T, T) \mid T; T.$$

$s \in \{\text{open}, L, R\}$ records an unresolved choice or the arm that made recorded progress, and g groups exclusive or jointly active siblings. $\llbracket T \rrbracket$ uses both arms of an open choice and only the selected arm thereafter. It interprets parallel and join with $\otimes$, and sequence with $\triangleright$. The join marker records a completed Merge and prevents the same group from being merged again.

T is complete when every pomset in $\llbracket T \rrbracket$ is empty. Lemma 1 ensures that the remaining workflow cannot contain both empty and nonempty pomsets. Set $\text{heads}(b:\mathcal{C}) = \{b\}$. An open choice makes both arms current, a selected choice makes its winner current, parallel and join make both arms current, and sequence makes its left operand current until completion and then its right. Completing a workflow call updates the work remaining at its leaf without deleting the recorded structure. Because sequence checks complete, it retains completed outcomes from its left operand, so an isolated completed leaf remains available for a later Fork or Restore. Joining removes g immediately but keeps its continuation behind a barrier until both arms complete. A workflow call is enabled when it is minimal in some pomset of $\llbracket T \rrbracket$. Disjoint workflow calls and unique context decomposition give every enabled x a unique current leaf $\text{leaf}_T(x) = b$. For $a \in \{\text{new}(x, d), \text{reuse}(x, d)\}$, define the paired execution update

$$\text{HStep}((G, T), a) = (G \parallel (b, a), T/(x, a)), \quad b = \text{leaf}_T(x),$$

where $G \parallel (b, a)$ appends progress record (b, a) , and $T/(x, a)$ removes the processed call from the current leaf and records any selected choice. Write $\text{HRes}((G, T), r)$ for iterating this update over a resolved word r .

$\text{addr}(T)$ is the set of branch and group IDs that an edit may currently name. It includes each current group and the current part of a sequence.

K maps checkpoint IDs to immutable earlier branch records. Write $\mathcal{C}_k = \text{contract}(K(k))$ for the remaining contract stored by checkpoint k . Σ_{ζ} is an immutable, versioned registry of typed edit rules. ρ records, for each workflow call, its action, authorization token, source region, scope, canonical tool request, and request digest. Write $q_{\text{act}}^{\rho}(x)$ for the action field of $\rho(x)$. For $d \in \text{im}(q_{\text{act}}^{\rho})$, let $\text{inv}_{\rho}(d)$ be the common canonical-request field of calls x with $q_{\text{act}}^{\rho}(x) = d$. Independently, $\ell(d) \in \mathcal{U}$ is the policy label certified for protected action d by the registered call model. For each current workflow call x , $\beta(x) = (j, \eta)$ records the rule entry j and version η used to check it. χ records processed workflow calls in order, including whether each created or reused an authorization record, and ver is the

state version. χ records authorization and workflow progress, while the remote service records completion and the returned value.

Checkpoint(k, b) creates a checkpoint for a current branch b under a fresh checkpoint ID k . A trusted transaction stores a protected copy of branch b 's remaining contract C_b , ordered progress record χ_b , relevant calls, and signed outcome requirements. The checkpoint check requires $C_b = \Gamma_b/\text{raw}(\chi_b)$, and the transaction advances ver without changing the current workflow, authorization records, β , the active rule version, or the authorization sequence. This internal transition changes H , so a rule update computed from the earlier record will fail its final check. Restore accepts only records created by this rule.

2) Well-formedness:

Definition 1 (Well-formed execution record). *H is well formed when the following groups hold.*

- 1) Structure. *G is acyclic, and branch and group IDs are globally unique across the recorded DAG and current workflow. Every ID that an edit may name has a unique context decomposition, and every node in $\text{heads}(T)$ is currently maximal. Every checkpoint names an immutable node and stores copies of its remaining contract and ordered progress record. Every original or edit-introduced outcome has exactly one signed policy authority. The controller preserves that authority when it carries or copies the outcome or stores it in a checkpoint.*
- 2) Identity and progress. *Workflow-call IDs and the j identifiers in β are unique, while workflow calls that refer to the same action agree on the trusted controller, scope, canonical request, digest, and source region. Processed workflow calls are downward closed in their pomset, consistent with every choice status, and absent from the remaining workflow.*
- 3) Current workflow. *$\text{dom}(\beta) = X_{[T]}$, and β maps every current workflow call x to exactly one entry j_x under the domain's current rule version η . Every current leaf $b:C$ equals its immutable registered contract Γ_b after removing the workflow calls recorded in χ_b ; the caller cannot replace it with a smaller outcome set. Every constructor in T is defined, so no complete sequence in any subtree is a strict prefix of another.*
- 4) Edit rules and domain. *Every edge and introduced contract follows one immutable edit rule at version ζ . Every current, checkpoint, or rule-introduced workflow call, action, source region, and scope belongs to ω . Protected actions, authorization logs, and policies of distinct domains are disjoint, and an edit never crosses a domain.*

The policy domain has exactly one active rule version, namely the version recorded in H .

3) Authorization and completion: The remaining parts of the execution record are policy version κ , a regular prefix-closed authorization policy $\mathcal{A} \subseteq \mathcal{U}^*$ over label alphabet $\mathcal{U}$, authorization log Δ , and queue out of requests awaiting release. Authorization records can only be appended. A completion update advances an authorization record's status and stores the returned value. Each authorization record binds its action,

immutable canonical tool request, creator workflow call, and authorization label. Each entry in χ points either to the authorization record it created or to the earlier record it reused, so (χ, Δ) determines retry and return lookup. $\text{lab}(\Delta)$ is the sequence of authorization labels on calls that create authorization records. Restore may reconstruct workspace data but leaves Δ unchanged.

The execution record $(\kappa, H, \Delta, \text{out}, \mathcal{A})$ is *well formed* when H is well formed, $\text{lab}(\Delta) \in \mathcal{A}$, each action has at most one authorization record, each processed call names the record it created or reused, the authorization log, request queue, and policy belong to ω , and the queue preserves the order of new authorizations. All definitions and results below quantify over such well-formed execution records.

C. Adversary and trusted boundary

The adversary controls Agent output, edit requests, tool arguments, copied workspace, retries, stale authorization tokens, and scheduling, including races between calls and edits. It cannot forge registries, the checkpoint directed acyclic graph (DAG), the record of which calls refer to the same action, the policy, or the authorization log. The policy authority signs policies, outcome requirements, and authorized removals. The trusted controller registers protected calls, actions, and authorization tokens, and it derives and checks each edit. The trusted controller checks the answer again before atomically changing the rules in force. The policy authority, trusted controller, protected registries and authorization log, proof verifier, and transaction mechanism form the trusted computing base (TCB). Within one policy domain, the TCB mediates every protected call and commits every state change and visible edit answer through a linearizable transaction. After an unexpected stop and restart, each transaction appears entirely before or entirely after its linearization point. Before the checker reads the execution record, the trusted controller may instantiate finitely many registered templates for tool calls. For tool request m , it computes canonical request $m^\circ = \text{canon}_\kappa(m)$, reuses an action certified for m° or allocates a fresh one, and signs $(x, j, d, \text{scope}, \text{digest}(m^\circ), \text{origin})$ into the authenticated registry. The resulting finite model $\mathcal{R}$ contains the workflow's protected calls and remains unchanged while the checker evaluates that execution record. The runtime rejects any call that does not match the registered model. The platform supplies workflow $\mathcal{W}$, finite model $\mathcal{R}$, edit rules, request-normalization rules, policy, checkpoints, authorization log, and queued requests in one protected execution record. From the execution record and an Agent request naming an edit and its objects, the controller derives the edited workflow, required outcomes, authorization decision for each call, safe execution set or rejection proof, and the rule entry and authorization token for each remaining call. Every derived object therefore comes from the execution record rather than Agent-supplied semantics.

Registration turns the protected-call behavior of a typed Agent workflow into the finite model $\mathcal{R}$. CheckReg verifies exact correspondence of outcomes, call order, actions, and

first-time authorization records. The model stores these traces as outcome-indexed runs $\text{BRuns}(\mathcal{R})$, together with a map λ from source traces. Write $\mathcal{W} \sqsubseteq_{\lambda} \mathcal{R}$ when λ is a bijection preserving outcomes, which calls refer to the same action, call order, outcome authority, and authorization-log updates. Definition 6 and lemma 13 give the exact finite check and proof. Registered contracts enter through CheckReg , and an idempotent or queryable remote service supplies authenticated completion records for its actions.

D. Security Objective

The security objective is fixed independently of the checker that will implement it. For an edit derived from the execution record, a safe execution set satisfies four requirements. First, it preserves every source outcome that is neither already satisfied nor explicitly authorized for removal, and every admitted execution belongs to the derived target workflow. Second, every admitted prefix is allowed by the policy after accounting for authorization records already present in the append-only log. Third, each admitted prefix extends to a complete admitted execution. Fourth, every outcome still compatible with an admitted prefix retains such a completion. The last requirement rejects the example in section II, where separately safe outcomes cannot remain safe under one set of runtime rules. Definition 2 states these requirements over executions labeled by outcome. Lemma 4 then proves that the checker computes the largest safe execution set.

IV. EXACT CHECKING OF EXECUTION EDITS

The checker first derives the exact edited workflow from the execution record and the requested edit. It then decides whether each workflow call creates or reuses an authorization record and removes precisely those complete executions that violate the security objective after some prefix.

A. Constructing the Edited Workflow

1) *Checking an edit rule:* Fork, Restore, and Merge have the six disjoint constructors of Edit_6 shown in table I. Checkpoint, the initial rule update, later rule registration, and protected calls use their own transitions.

An Agent request u names an edit kind, current object IDs, and edit-rule ID σ . Registry Σ_{ζ} records the expected source and checkpoint patterns for σ , any workflow steps the edit adds, the source region for each copy, and the authority that signs each introduced outcome.

The checker constructs explicit preservation maps for every region that the rule carries or copies. Consider a source contract $\mathcal{C}$, target region $\mathcal{D}$, and parent map $\mu : X_{\mathcal{D}} \rightarrow X_{\mathcal{C}}$. Write $\mathcal{C} \preceq_{\mu} \mathcal{D}$ when μ is a global bijection and preserves actions and source regions. The relation also requires a surjection $\pi : O_{\mathcal{D}} \rightarrow O_{\mathcal{C}}$ that satisfies the conditions below for every target outcome v .

$$\begin{aligned} x \in X_{p_v} &\iff \mu(x) \in X_{p_{\pi(v)}}, \\ x \prec_v y &\iff \mu(x) \prec_{\pi(v)} \mu(y). \end{aligned}$$

Because μ is one global bijection and preserves which outcomes contain each call, it cannot split a source call shared by several outcomes into separate target calls.

For candidate T' , let R_u index the disjoint regions carried or copied inside the replaced subtree. $\mathcal{P}_u = \{(\mathcal{C}_r, \mathcal{D}_r, \mu_r, \pi_r)\}_{r \in R_u}$ collects these preservation maps. A separate identity map verifies that the surrounding workflow preserves syntax, outcomes, workflow calls, actions, source regions, and order. Writing X_{ctx} for its workflow calls and $X_{\text{added}(u)}$ for workflow calls added by the rule, the checker verifies the following disjoint cover.

$$X_{[T']} = X_{\text{ctx}} \uplus \biguplus_{r \in R_u} X_{\mathcal{D}_r} \uplus X_{\text{added}(u)}.$$

The edit rule and surrounding workflow induce $\text{pr}_r^u : O_{[T']} \rightarrow O_{\mathcal{D}_r}$. For choice, this map is defined only in the arm containing r , while product and sequence select the corresponding indexed component. Define

$$\text{EditedRequired}_u = \{(r, o) \mid r \in R_u, o \in O_{\mathcal{C}_r}\},$$

$$\text{Covers}_u(t, r, o) \iff \exists v \in O_{\mathcal{D}_r}. \text{pr}_r^u(t) = v \wedge \pi_r(v) = o.$$

Let $O_{\text{unchanged}}$ be the outcomes outside the edited region, and let $\iota_{\text{ctx}}^u : O_{\text{unchanged}} \hookrightarrow O_{[T']}$ be their unchanged embedding into the target. Define the outcomes that remain required and the target outcomes that cover them by

$$\begin{aligned} \text{StillRequired}_u &= \text{EditedRequired}_u \\ &\quad \uplus (\{\text{ctx}\} \times O_{\text{unchanged}}), \\ \text{Covers}_u(t, (\text{ctx}, o)) &\iff \\ t &= \iota_{\text{ctx}}^u(o). \end{aligned} \tag{4}$$

For $a = (r, o) \in \text{EditedRequired}_u$, $\text{Covers}_u(t, a)$ denotes the relation above. The checker also requires $\forall a \in \text{StillRequired}_u. \exists t \in O_{[T']}. \text{Covers}_u(t, a)$: every required source outcome has a complete target outcome, shared workflow calls remain shared, and inherited calls remain distinct from steps added by the edit rule.

The edit-rule check uses the following judgment.

$$\Sigma_{\zeta}; H \vdash_{\text{edit}} u \Rightarrow (T', \mathcal{P}_u)$$

It holds exactly when every named object belongs to $\text{addr}(T)$, every introduced object belongs to ω , the source pattern matches, the preservation maps and disjoint cover hold, every still-required outcome is covered, every removal carries the proper authority, and $[T']$ is defined. The policy authority may authorize removal of the unselected arm of a typed MergeSelect or the current branch of RestoreReplace , using the signed sets defined below.

2) *Required-outcome preservation:* The registered edit rule determines every outcome that the edit must account for. Fork accounts for both copies, Restore for the current and checkpoint branches, MergeSelect for the selected and unselected arms, and MergeJoin for both arms. Outcomes outside the edited region remain unchanged. Definition 4 defines the outcomes required before the edit, those already satisfied, and those whose

removal the policy authority signed. The checker accepts the edit rule when the target-derived set from equation (4) satisfies

$$\text{StillRequired}_u = \text{RequiredBefore}(H, u) \setminus (\text{Satisfied}_{H,u} \uplus \text{AuthorizedRemoval}_{H,u}),$$

and hence the exact disjoint partition

$$\begin{aligned} \text{RequiredBefore}(H, u) &= \text{StillRequired}_u \\ &\uplus \text{Satisfied}_{H,u} \\ &\uplus \text{AuthorizedRemoval}_{H,u}. \end{aligned}$$

This partition accounts for every source outcome and prevents an Agent request from silently dropping one. The preservation maps retain actions, source regions, and order, while the rule update retains the records for satisfied outcomes and authorized removals.

Let $\mathcal{E}[-]$ be the current workflow with one editable region replaced by a hole. carry preserves branch, outcome, and workflow-call IDs, pomset order, actions, signed outcome authorities, base contracts, ordered progress records, and nested choices. clone_r assigns fresh branch, outcome, and workflow-call IDs while preserving pomset order, actions, scope, digest, source regions, and the authority for each outcome. Each copied branch starts from its registered remaining contract with an empty progress record. Branches added by an edit rule likewise start from their registered contract and an empty progress record. The contracts $E_\sigma^L, E_\sigma^R, E_\sigma^J$ are additional workflow steps derived from the rule registry. Define

$$L_\sigma = \text{clone}_L(\mathcal{C}) \triangleright E_\sigma^L, \quad R_\sigma = \text{clone}_R(\mathcal{C}) \triangleright E_\sigma^R.$$

New identifiers and the target rule version η^+ are deterministically allocated from $(\omega, \zeta, \text{ver}, u, \text{role})$. role identifies which operation argument receives each new ID. Write $\text{Copy}_r(\mathcal{C})$ for $(\mathcal{C}, \text{clone}_r(\mathcal{C}), \mu_r, \pi_r)$, and $\text{Keep}(\mathcal{C})$ for $(\mathcal{C}, \text{carry}(\mathcal{C}), \mu, \pi)$. Each abbreviation asserts the corresponding $\preceq_\mu$ relation. For a current workflow T_0 , $\text{Keep}(T_0)$ abbreviates $\text{Keep}(\llbracket T_0 \rrbracket)$ and has target contract $\llbracket \text{carry}(T_0) \rrbracket$. Every b or g named below must lie in $\text{addr}(T)$.

In an open choice, the first processed workflow call atomically selects its arm and disables later calls from the other arm, whether that first call creates or reuses an authorization record. Consequently, $\text{MergeSelect}(g, w, \sigma)$ is valid precisely while the unselected arm has made no recorded progress. MergeJoin replaces the current $\parallel_g$ node with join, removes g from the set of IDs an edit may name, and prevents the same pair from being joined again.

Every edit replaces all runtime rules for its policy domain, and $\beta(x)$ uses $\eta^- = \eta$ for every current call x .

3) *Preparing calls for the new rules:* The execution-edit judgment consists exactly of one row of table I applied inside the well-typed workflow context $\mathcal{E}$, followed by the state update below. Let e_u contain the edit record and exactly the copied, added, and group nodes prescribed by T' ; carried node IDs remain unchanged. For every current target workflow call x , the trusted controller deterministically creates a rule entry j_x^+ and authorization token h_x^+ . The updated record ρ' preserves

the action, source region, scope, digest, and retry information. Define

$$\begin{aligned} \mathcal{H}_u^+ &= \{(x, j_x^+, h_x^+) \mid x \in X_{[T']}\}, \\ \beta'(x) &= (j_x^+, \eta^+). \end{aligned}$$

The new authorization tokens are inactive and inaccessible to the caller before the rule update takes effect. The complete judgment is shown below.

$$\begin{aligned} \Sigma_\zeta; H \vdash u \Downarrow (H', \mathcal{C}_{H,u}, \eta^-, \eta^+), \\ H' = (\omega, G \cdot e_u, T', K, \Sigma_\zeta, \rho', \beta', \chi, \text{ver} + 1, \eta^+), \\ \mathcal{C}_{H,u} = \llbracket T' \rrbracket, \quad \eta^- = \eta. \end{aligned} \tag{5}$$

Here $G \cdot e_u$ retains earlier progress, outcome-authority records, and existing nodes, and initializes each copied or added branch from its remaining registered contract with inherited or rule-signed outcome authority and no progress. Thus an edit preserves K, Σ_ζ, χ , appends one record of the edit and its target nodes, creates a new rule entry and authorization token for every current call, and moves the domain to η^+ . Table constructors determine new group modes, while carried nested modes remain unchanged. A nonexistent checkpoint, wrong group mode, nonmember winner, mismatched rule pattern, unauthorized removal, or undefined composition produces Invalid. Independent domains step by asynchronous product, and each domain replaces all of its runtime rules at once.

Lemma 2 (Exact edit derivation and atomic rule update). *For well-formed H , the judgment in equation (5) is deterministic. If it derives H' , then H' is well formed. It preserves required outcomes and their signed authorities, retaining a record for every satisfied outcome and every still-required source outcome except a signed authorized removal. The surrounding-workflow identity map and family $\mathcal{P}_u$ preserve which calls refer to the same action, source regions, the authority for each outcome, and causal order. For every $a \in \text{StillRequired}_u$, some complete target outcome t satisfies $\text{Covers}_u(t, a)$, including every unchanged outcome outside the edited region. Every current target workflow call x has $\beta'(x) = (j_x^+, \eta^+)$ and an authorization token for j_x^+ .*

Proof sketch. The six cases use the copy/keep bijections and surrounding-workflow identity map to preserve action sharing, source regions, and order. They also construct a target outcome satisfying each required Covers_u relation. The common state update establishes well-formedness and sets $\beta'(x) = (j_x^+, \eta^+)$ for every current target workflow call. Section A-D1 gives the full argument. $\square$

Recall the authorization-label alphabet $\mathcal{U}$ and regular prefix-closed policy $\mathcal{A} \subseteq \mathcal{U}^*$. For authorization log Δ , $\text{lab}(\Delta) \in \mathcal{A}$ is its authorization sequence and D_Δ is the set of actions that already have authorization records. Fix $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$ and write $\text{state}(\Theta) = (\kappa, H, \Delta, \text{out}, \mathcal{A})$. Derive is partial; whenever the initial rule update, an execution edit, or a later registered extension is valid, it produces a unique derived record. When defined, write $\text{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. Root derivation additionally requires that $\text{CheckReg}(\mathcal{W}, \mathcal{R})$

TABLE I
THE SIX EXECUTION FORMS OF FORK, RESTORE, AND MERGE. $E_\sigma^\bullet$ IS DERIVED FROM THE REGISTERED EDIT RULE IN Σ_ζ .

Operation u	Required current shape	Derived target shape	Checked preservation
ForkChoice(b, σ)	$\mathcal{E}[b:C]$	$\mathcal{E}[b_L:L_\sigma \oplus_g^{\text{open}} b_R:R_\sigma]$	$\{\text{Copy}_L(C), \text{Copy}_R(C)\}$
ForkParallel(b, σ)	$\mathcal{E}[b:C]$	$\mathcal{E}[b_L:L_\sigma \parallel_g b_R:R_\sigma]$	$\{\text{Copy}_L(C), \text{Copy}_R(C)\}$
RestoreReplace(b, k, σ)	$\mathcal{E}[b:C], k \in \text{dom } K$	$\mathcal{E}[b_k:\text{clone}_k(C_k)]$	$\{\text{Copy}_k(C_k)\}$; current branch removed
RestoreLive(b, k, σ)	$\mathcal{E}[b:C], k \in \text{dom } K$	$\mathcal{E}[b:\text{carry}(C) \parallel_g b_k:\text{clone}_k(C_k)]$	$\{\text{Keep}(C), \text{Copy}_k(C_k)\}$
MergeSelect(g, w, σ)	$\mathcal{E}[T_L \oplus_g^* T_R], s \in \{\text{open}, w\}$	$\mathcal{E}[\text{carry}(T_w); b_J:E_\sigma^J]$	$\{\text{Keep}(T_w)\}$; other arm removed
MergeJoin(g, σ)	$\mathcal{E}[T_L \parallel_g T_R]$	$\mathcal{E}[\text{join}(\text{carry}(T_L), \text{carry}(T_R)); b_J:E_\sigma^J]$	$\{\text{Keep}(T_L), \text{Keep}(T_R)\}$

accept. An execution edit has $(\kappa_u, \mathcal{A}_u) = (\kappa, \mathcal{A})$ and uses the unique execution-edit judgment. $\text{Extend}(\xi)$ adds authenticated edit-rule, call, outcome-authority, and policy entries while preserving the current workflow, its progress, and all earlier authorizations; section A-D2 gives the judgment. Let ρ_u be the target registry in H_u , let $O_{H,u} = O_{C_u}$, and use $H' = H_u$ and $\rho' = \rho_u$ where earlier notation is convenient. The execution sets below use Θ , the edit-rule registry, ρ_u , and target policy $\mathcal{A}_u$. Before the rules change, the trusted controller checks the complete record again, including queued requests out, so a concurrent release makes an earlier answer stale. We abbreviate the fixed parameters by writing H, u .

B. New and Reused Authorizations

Function $\text{Resolve}_{\Delta}^{\rho_u}$ keeps every workflow call and annotates it as creating a new authorization or reusing an earlier one. Starting with $D_0 = D_\Delta$, it maps a call sequence $w = x_1 \cdots x_n$ to annotated sequence $z_1 \cdots z_n$. For $d_i = q_{\text{act}}^{\rho_u}(x_i)$,

$$z_i = \begin{cases} \text{new}(x_i, d_i), & d_i \notin D_{i-1}, \quad D_i = D_{i-1} \cup \{d_i\}; \\ \text{reuse}(x_i, d_i), & d_i \in D_{i-1}, \quad D_i = D_{i-1}. \end{cases} \quad (6)$$

auth keeps the authorization label of each new authorization and contributes nothing for reuse.

$$\text{auth}(\text{new}(x, d)) = \ell(d), \quad \text{auth}(\text{reuse}(x, d)) = \epsilon.$$

For annotated z , $\text{raw}(z)$ drops the new/reuse annotation and action while preserving the ordered workflow-call IDs. A Retry repeats the same canonical request and adds no workflow call. Restored workflow calls that share an action remain distinct. The first call creates the authorization record, and later calls reuse it.

Lemma 3 (New/reuse classification). $\text{Resolve}_{\Delta}^{\rho_u}$ is deterministic, preserves length, and gives the same annotations on every shared prefix. Among calls annotated as new, each action occurs at most once outside D_Δ , and dropping the annotations gives exactly the input sequence. Moreover, if $\text{Resolve}_{\Delta}^{\rho_u}(rv) = zv'$, then $z = \text{Resolve}_{\Delta}^{\rho_u}(r)$ and $v' = \text{Resolve}_{\Delta_z}^{\rho_u}(v)$, where Δ_z extends Δ by the new authorizations in z .

Proof. Induction on input length shows that the function emits one prefix-determined annotation per input symbol and updates D_i exactly when it creates a new authorization. Set membership ensures at most one new authorization per action. Both cases

retain x_i . Splitting after r gives the stated composition equation. $\square$

Thus a retry changes neither the processed-call record nor the authorization sequence, although its successful API reply is observable as $\text{Return}(t, [v]_\cong)$. A copied workflow call that reuses an authorization record advances the processed-call record without changing the authorization sequence.

C. Safe Completion After Every Prefix

1) *Computing all safe behavior:* For each derived outcome $o \in O_{H,u}$, define all of its complete executions and the subset allowed by policy.

$$R_o(H, u) = \{\text{Resolve}_{\Delta}^{\rho_u}(w) \mid w \in \text{Lin}(p_o)\}, \quad (7)$$

$$M_o(H, u) = \{z \in R_o(H, u) \mid \text{lab}(\Delta)\text{auth}(z) \in \mathcal{A}_u\}.$$

For an annotated prefix z , let

$$\text{Compat}(z) = \{o \mid \text{raw}(z) \preceq_{\text{pref}} w \text{ for some } w \in \text{Lin}(p_o)\}.$$

$\text{PreservesRequired}(H, u, \hat{B})$ means that every outcome in StillRequired_u is covered by some target outcome represented in $\hat{B}$:

$$\forall a \in \text{StillRequired}_u. \exists (t, y) \in \hat{B}. \text{Covers}_u(t, a).$$

Definition 2 (Safe execution set). A safe execution set is a pair $(L, \hat{B})$, where L contains allowed partial executions and $\hat{B}$ contains allowed complete executions labeled by outcome, satisfying the conditions below.

- 1) $\hat{B} \neq \emptyset$;
- 2) workflow and outcomes: $\text{PreservesRequired}(H, u, \hat{B})$, $\hat{B} \subseteq \biguplus_{o \in O_{H,u}} \{o\} \times R_o(H, u)$, and $L \subseteq \text{Pref}(\bigcup_{o \in O_{H,u}} R_o(H, u))$;
- 3) policy: $\text{lab}(\Delta)\text{auth}(z) \in \mathcal{A}_u$ for every $z \in L$;
- 4) some completion after every prefix: $L = \text{Pref}(\pi_2 \hat{B})$; and
- 5) every compatible outcome remains completable: for every $z \in L$ and $o \in \text{Compat}(z)$, some $(o, y) \in \hat{B}$ extends z .

Checking each outcome separately is insufficient. After a partial execution shared by several outcomes, every outcome still compatible with that partial execution must retain a policy-safe completion. The operator below removes exactly the complete executions that violate this condition. Put

$$\hat{B}_0 = \biguplus_{o \in O_{H,u}} \{o\} \times M_o.$$

For $\hat{B} \subseteq \hat{B}_0$, define the monotone operator

$$\hat{\Phi}(\hat{B}) = \left\{ (o, y) \in \hat{B}_0 \mid \begin{array}{l} \forall z \preceq_{\text{pref}} y. \forall o' \in \text{Compat}(z). \\ \exists (o', y') \in \hat{B}. z \preceq_{\text{pref}} y' \end{array} \right\}. \quad (8)$$

Set $\hat{B}_{i+1} = \hat{\Phi}(\hat{B}_i)$. Since $\hat{B}_1 \subseteq \hat{B}_0$, repeated application removes executions until the finite set stops changing:

$$\begin{aligned} \hat{B}_{H,u}^\dagger &= \nu \hat{B}. \hat{\Phi}(\hat{B}) = \bigcap_{i \geq 0} \hat{B}_i, \\ B_{H,u}^\dagger &= \pi_2 \hat{B}_{H,u}^\dagger, & W_{H,u}^\dagger &= \text{Pref}(B_{H,u}^\dagger). \end{aligned}$$

Definition 3 (Safe execution edit).

$$\text{SafeEdit}(\Theta) \iff \begin{cases} \hat{B}_{H,u}^\dagger \neq \emptyset, & \text{Derive}(\Theta) \text{ is defined;} \\ \text{false}, & \text{otherwise.} \end{cases} \quad (9)$$

At $z = \epsilon$, nonemptiness covers every required outcome. The existential quantifier chooses a call order, and the universal condition keeps every compatible outcome completable after every prefix.

Lemma 4 (Largest safe execution set). *A safe execution set exists iff $\text{SafeEdit}(\Theta)$. When the edit is safe, $(W_{H,u}^\dagger, \hat{B}_{H,u}^\dagger)$ is a safe execution set, and every safe execution set $(L, \hat{B})$ satisfies $\hat{B} \subseteq \hat{B}_{H,u}^\dagger$ and $L \subseteq W_{H,u}^\dagger$.*

Proof. The workflow, outcome, and policy conditions place every safe execution set's complete executions inside $\hat{B}_0$. The final condition makes this set post-fixed, so monotonicity and finite Knaster–Tarski place it inside $\hat{B}^\dagger$, and projected prefixes give $L \subseteq W^\dagger$. Conversely, $\hat{B}^\dagger = \hat{\Phi}(\hat{B}^\dagger)$ keeps every compatible outcome completable. At the empty prefix, this condition and the checked coverage witnesses from lemma 2 establish PreservesRequired. The inclusion $\hat{B}^\dagger \subseteq \hat{B}_0$ ensures that complete executions follow the edited workflow and policy, while prefix closure of $\mathcal{A}_u$ makes every allowed prefix safe. Finally, $W^\dagger = \text{Pref}(\pi_2 \hat{B}^\dagger)$ gives a completion after every allowed prefix. Hence the fixed point is a safe execution set exactly when nonempty. $\square$

$\text{MaxSafe}(\Theta; L, \hat{B})$ means that $(L, \hat{B})$ is safe and contains every other safe execution set. When such a pair exists, write its unique value as $\text{SafeBehavior}(\Theta)$. Lemma 4 proves that this witness exists exactly when $\hat{B}_{H,u}^\dagger \neq \emptyset$ and then equals $(W_{H,u}^\dagger, \hat{B}_{H,u}^\dagger)$.

2) *Why Outcome-by-Outcome Checking Fails:* Section II introduced the contract $X \otimes (Y \oplus Z)$ under policy $\text{Pref}\{yx, xz\}$. For its annotated calls, write $\bar{x} = \text{new}(x_o, d_x)$, $\bar{y} = \text{new}(y_o, d_y)$, and $\bar{z} = \text{new}(z_o, d_z)$. The candidate family initially contains $\bar{y}\bar{x}$ and $\bar{x}\bar{z}$. The first fixed-point iteration removes $\bar{x}\bar{z}$. The second iteration removes $\bar{y}\bar{x}$, because after the first removal the empty prefix no longer has a completion for the $X \otimes Z$ outcome. The empty fixed point is the formal reason the exact checker rejects.

Proposition 1 (Exact correspondence with generalized non-blocking). *For nonempty $\hat{B} \subseteq \hat{B}_0$, let $\mathcal{T}_{\hat{B}}$ be the prefix trie*

of $L_{\hat{B}} = \text{Pref}(\pi_2 \hat{B})$. Mark state z by possible_o exactly when $o \in \text{Compat}(z)$, and mark state y by done_o exactly when $(o, y) \in \hat{B}$. Then

$$\begin{aligned} \hat{B} &\subseteq \hat{\Phi}(\hat{B}) \\ \iff \forall o \in O_{H,u}. \mathcal{T}_{\hat{B}} \text{ is } (\text{possible}_o, \text{done}_o)\text{-nonblocking.} \end{aligned}$$

Consequently, if $\hat{B}^\dagger$ is nonempty, it is the largest nonempty completion subfamily of $\hat{B}_0$ satisfying these outcome-specific nonblocking conditions. If it is empty, no nonempty subfamily satisfies the conditions. Ordinary marker nonblocking of the unaugmented projected language is strictly weaker.

Proof. From reachable prefix z , a done_o -marked state is reachable exactly when some $(o, y) \in \hat{B}$ extends z . Quantifying over exactly the states marked possible_o is therefore the post-fixed-point condition. Greatestness follows from the greatest-fixed-point construction. For strictness, the preceding instance has the marker-nonblocking projected pair $(\text{Pref}\{\bar{y}\bar{x}, \bar{x}\bar{z}\}, \{\bar{y}\bar{x}, \bar{x}\bar{z}\})$, but its indexed fixed point is empty. $\square$

Proposition 1 relates the fixed point to generalized non-blocking after the controller has derived the outcome-specific completion conditions [16], [17]. The controller derives those conditions from the requested edit, authorized removals, actions, authorization records, and the execution record. It also decides whether each protected call creates or reuses an authorization, computes every safe continuation or a rejection proof, and puts an accepted result into effect atomically.

3) *Rejection Proofs:* Each removed (o_{rem}, y) records its rank i , a prefix $z \preceq y$, and uncovered $o_{\text{miss}} \in \text{Compat}(z)$. Induction shows that every post-fixed P is contained in every $\hat{B}_i$. An empty final family therefore proves that no safe execution set exists. Before the rule update, the trusted controller recomputes both the removal order and the remaining completions.

4) *Checking Again After a Permitted Prefix:* To show that the checker can be applied again after each permitted partial execution, fix $H, u, \mathcal{A}_u$ after deriving $\mathcal{C}$, and write $W^\dagger(\mathcal{C}, \Delta)$ and $\hat{B}^\dagger(\mathcal{C}, \Delta)$ with those parameters suppressed.

Lemma 5 (Rechecking after a prefix). *If $z \in W^\dagger(\mathcal{C}, \Delta)$, then $\text{Resolve}_{\Delta}^{\rho'}(\text{raw}(z)) = z$, $\text{lab}(\Delta_z) = \text{lab}(\Delta)\text{auth}(z)$, and*

$$\begin{aligned} W^\dagger(\mathcal{C}, \Delta)/z &= W^\dagger(\mathcal{C}/\text{raw}(z), \Delta_z), \\ \hat{B}^\dagger(\mathcal{C}, \Delta)/z &= \hat{B}^\dagger(\mathcal{C}/\text{raw}(z), \Delta_z), \end{aligned}$$

where $\hat{B}/z = \{(o, v) \mid (o, zv) \in \hat{B}\}$.

Proof sketch. Streaming resolution recovers the original calls, gives the authorization-log equality, and makes the candidate executions after z identical on both sides. Removing prefix z from $\hat{B}^\dagger$ gives a post-fixed family for the next check, while adding z to any such family gives one for the original check. Greatestness yields the fixed-point equality, and $\text{Pref}(B)/z = \text{Pref}(B/z)$ yields the generated-language equality. The full proof is in section A-D3. $\square$

Thus every accepted prefix retains a safe completion for every outcome that remains compatible with that prefix.

5) *Checker Outputs*: The checker returns Invalid when derivation fails, Reject with the ordered reasons for removing every execution when the fixed point is empty, and Accepted with the largest safe execution set otherwise. Write $\text{Check}(\Theta)$ for this deterministic output. Each answer includes a digest of the execution record that was checked. Section A-C gives the exact returned data and verification rules. Proposition 2 proves an output-sensitive $O(D + n\Lambda + |\mathcal{G}_{\text{cov}}| + V_B \log(2 + V_B))$ -time, $O(D + \Lambda + |\mathcal{G}_{\text{cov}}|)$ -space bound. Exponential growth occurs only in explicitly emitted linearizations and prefix-required-outcome support pairs.

V. RUNTIME ENFORCEMENT

The safety checker produces either a rejection proof or the largest safe family of complete executions. The trusted controller turns a nonempty family into rules checked before each tool call. It builds a finite automaton for the allowed partial executions, records which version of the execution record was checked, and atomically puts the new rules into effect. The protocol orders every overlapping protected call either before or after the atomic rule update. The proof compares this implementation with an ideal machine that puts an accepted result into effect in one atomic step and admits exactly its resolved prefixes. The ideal transition system and the field-level implementation rules appear in section A-H and table III. This section gives the automaton construction, the invariant after the rules change, and the atomic update used by the main theorem.

A. Turning Safe Executions into Runtime Rules

Removing outcome labels from the largest safe execution set gives the finite pair $(W, B) = (W_{H,u}^\dagger, B_{H,u}^\dagger)$. For $L/z = \{v \mid zv \in L\}$, let $q_z = (W/z, B/z)$ be the automaton state after prefix z .

$$\begin{aligned} Q_{W,B} &= \{q_z \mid z \in W\}, \\ q^0 &= q_\epsilon = (W, B), \\ q_z &\xrightarrow{a} q_{za} \iff za \in W. \end{aligned} \quad (10)$$

Call this deterministic automaton $\text{Can}(W, B)$. Every state is reachable and is marked exactly when $\epsilon \in B^\dagger/z$. Each transition symbol a contains workflow call x , action d , and its new or reuse mode, so one workflow call cannot use another call's rule entry.

Lemma 6 (Exact finite enforcement). *The generated and marked languages of the automaton in equation (10) are exactly $(W_{H,u}^\dagger, B_{H,u}^\dagger)$. The automaton is the smallest deterministic marked partial automaton for this pair up to state renaming.*

Proof. Induction on z shows that the automaton reaches $(W^\dagger/z, B^\dagger/z)$, enables a iff $za \in W^\dagger$, and marks the reached state iff $z \in B^\dagger$. Distinct states differ on a generated or marked suffix and must remain distinct. The automaton is the paired-language derivative construction [20]. $\square$

B. Atomic Rule Update

Before the rules change, the trusted controller re-derives the edit and recomputes the fixed point from the execution record. If any protected call has committed since the candidate was computed, the candidate is stale and changes nothing. Otherwise, one policy-domain transaction closes the old rule version, activates the new automaton, replaces the rule entry and authorization token for every current workflow call, and only then exposes the successful edit answer. Protected calls verify their workflow call, action, canonical request, authorization token, and enabled automaton transition in the same transaction that records progress. A new call creates one authorization record and queues the canonical request. A reuse call advances the workflow by linking to the existing record without extending the authorization sequence.

a) Invariant after the rule update: We write $\text{AgentSec}(S; c, r)$ when five conditions hold: (1) the workflow now in force and its required outcomes come from the checked edit; (2) the stored nonempty execution family is the checker's largest safe family; (3) the workflow, authorization log, queue, and checkpoints match the executed prefix r ; (4) the automaton is in exactly the state reached after r ; and (5) exactly one rule version is active, with one rule entry and authorization token for every remaining workflow call. For a submitted tool request, one transaction verifies the workflow call, action, canonical request, authorization token, active rule version, and enabled automaton transition before recording any progress. The same transaction either creates the authorization record and queues the request, or records reuse of an existing authorization record and its return value.

b) Final check: For current state S , write $\Theta_S(u) = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$ for the safety-check instance obtained from its current components. Write $\text{Ready}(S, u, Y)$ when the service re-derives u from the current execution record, recomputes the same nonempty fixed point and automaton carried by Y , confirms that the record has not changed since Y was computed, confirms that the old rule version is current, and confirms that the new version is inactive. If these checks pass, one transaction closes the old version, activates the new automaton, replaces every current rule entry and authorization token, and then exposes the successful edit answer. A changed state makes the candidate stale and forces a new check. The complete state updates, retries, request release, completion updates, and restart rules appear in section A-I and table III.

Lemma 7 (Edit-call serialization). *In every reachable well-formed runtime state, a visible edit answer and a protected call in the same policy domain have a unique order at the atomic rule update. If the call commits first, it changes the execution record, so the earlier result cannot take effect. If the rule update commits first, the old call cannot resolve new or reuse. Steps in independent domains commute by product semantics.*

Proof. If the call commits first, its recorded progress changes the execution record. If the rule update commits first, it closes the old rule version, so the old authorization token

fails. Atomicity excludes a partial third outcome. Independent domains commute. $\square$

C. Trust Boundary

The TCB is the trusted boundary defined in section III. Search, minimization, and candidate preparation remain untrusted because the final check recomputes them. The next section states the exactness and runtime guarantees obtained at this boundary.

VI. FORMAL GUARANTEES

The theorem connects derivation from the execution record, exact checking, finite enforcement, the atomic rule update, and every later runtime step. Its central clause says that the checker accepts if and only if a safe runtime implementation exists. The other clauses establish faithful registration, the information required for an exact answer, repeated use, and runtime correctness.

When defined, write $\delta = \text{Derive}(\Theta) = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. A *safe runtime implementation* is an automaton M together with complete executions $\hat{B}_M$ labeled by outcome. The partial and complete executions of M must form a safe execution set as defined in definition 2. The atomic rule update activates this automaton at η^+ , closes η^- , and replaces the rule entries and authorization tokens for all target calls.

a) *What the checker must know:* Write $V = \text{Sec}(S; c, r) = (\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C})$ for four parts of the checked record: (1) the workflow and its required outcomes, (2) which calls refer to the same action, (3) earlier authorizations and call progress, and (4) which rules govern a call that overlaps the atomic rule update. Suppose a checker receives only some function $f(V)$ of this record. The function is exact if any two records that it makes indistinguishable always give the same answer to every common decision or rule-update request. Write $\text{Exact}(f)$ for this property. Internal identifiers may be renamed consistently because their spelling cannot affect the answer. For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, Keep_J gives the checker exactly the parts named by J . Section A-F gives one record pair for each part and a fifth pair for the correspondence between earlier authorizations and actions.

b) *Required-outcome preservation:*

$$\text{PreservesRequired}(H, u, \hat{B}) \iff \forall a \in \text{StillRequired}_u. \exists (t, y) \in \hat{B}. \text{Covers}_u(t, a).$$

Thus every required source outcome appears in an accepted target outcome.

c) *Visible runtime behavior:* $\mathcal{O}_{\text{api}}$ includes edit answers tied to the checked record, edits put into effect, protected-call answers, released requests, completion updates, and returned values. It hides internal computation, Checkpoint, stale candidates, and restart bookkeeping.

$$\text{obs}_{\text{api}}(\ell) = \begin{cases} \ell, & \ell \in \mathcal{O}_{\text{api}}; \\ \tau, & \text{otherwise.} \end{cases}$$

Write $S \approx_{\mathcal{O}_{\text{api}}}^{\text{di}} I$ when S and I are related by a divergence-insensitive weak bisimulation under obs_{api} . $\mathcal{B}$ matches ideal and runtime states on AgentSec and these visible fields. Section A-L gives the full relation.

Theorem 1 (Faithful Registration). *For finite-state $\mathcal{W}$ and finite $\mathcal{R}$, $\text{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts iff stored λ witnesses $\mathcal{W} \sqsubseteq_{\lambda} \mathcal{R}$. On acceptance, the source workflow's protected-call traces and $\text{BRuns}(\mathcal{R})$ are order-isomorphic and preserve outcomes, actions, new/reuse, the safety answer, and the largest safe execution set. Malformed registrations fail before startup.*

For the remaining results, assume accepted registration, trusted checking of every protected call, and linearizable transactions that remain atomic after a restart within each policy domain. Checkpoint(k, b) succeeds exactly when k is fresh, b is current, and the stored work matches the branch's remaining registered work: $\mathcal{C}_b = \Gamma_b / \text{raw}(\chi_b)$. A successful Checkpoint preserves AgentSec without changing the current workflow or any earlier authorization. Let $\text{Ext} = \{\text{Extend}(\xi) \mid \xi \text{ is verified}\}$. The theorems below hold for every $u \in \text{Edit}_6 \uplus \text{Ext}$ at every finite reachable AgentSec state.

Theorem 2 (Exact Safety Checking). *The ideal and runtime machines return the same mutually exclusive and exhaustive answer for $\Theta = \Theta_S(u)$. Undefined derivation gives state-preserving Invalid. For defined $\delta = \text{Derive}(\Theta)$, $\hat{B}_{H,u}^+ = \emptyset$ gives state-preserving Reject with no safe runtime implementation. Otherwise, for $Y = \text{Check}(\Theta)$, nonempty $\hat{B}_{H,u}^+$ existence of a safe runtime implementation, $\text{Ready}(S, u, Y)$, and a successor that puts the edit into effect while preserving AgentSec are equivalent. The largest execution family satisfies PreservesRequired, and the derived record identifies every already-satisfied source outcome and every authorized removal.*

Theorem 3 (Necessary Record Information). *The full record is sufficient. If any one of the four parts is omitted, two records become indistinguishable even though the correct answer differs.*

$$\text{Exact}(\text{Keep}_J) \iff J = \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}.$$

The four component-labeled rows in table II witness omission of $\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}$. The Drop_{ra} row shows that retaining authorization records and actions without recording which ones correspond is also insufficient. In particular, a checker is not exact if it forgets which calls or authorization records refer to the same action, or if it sees only the target workflow proposed by the caller. Generalized nonblocking can serve as an exact solver only after the trusted controller derives the workflow and outcome-specific completion conditions.

Theorem 4 (Repeated-Use Safety). *Every enabled event preserves AgentSec, so the theorem applies after every finite reachable sequence of edits, registered extensions, protected calls, authorization updates, rule updates, stops, and restarts.*

Theorem 5 (Runtime Correctness). *$\mathcal{B}$ is a divergence-insensitive weak bisimulation under obs_{api} , so every pair related by $\mathcal{B}$ satisfies $S_0 \approx_{\mathcal{O}_{\text{api}}}^{\text{di}} I_0$. The observation records*

edit answers tied to the checked record, new authorization data, and returned values.

d) Proof roadmap: Registration reflection follows by enumerating the finite protected-call traces and checking their order- and identity-preserving correspondence. The edit-rule checks and greatest-fixed-point argument prove exact checking. Every safe execution set is contained in the computed family, and a nonempty computed family yields a safe runtime implementation. Five explicit pairs of execution records prove the information result. Induction over the runtime rules proves repeated use. A case analysis over edits, protected calls, races with the rule update, returns, and restarts establishes the weak bisimulation. The appendix gives the complete case proofs.

VII. MECHANIZATION AND EXECUTABLE VALIDATION

Six Lean modules pass `--trust=0` and mechanize the finite linear-contract core used by the paper. `exact_checker_iff_realization` proves that the core executable checker accepts exactly when a `ValidRealization` exists. `registered_workflow_exact_admission` proves registration acceptance, exact mediation of registered calls, and equivalence between checker acceptance and a valid realization for the current execution record. `six_edit_derivation_exact` proves that executable workflow editing is sound and complete for the six edit constructors. `exact_edit_installs_trace_safe_monitor` constructs an atomic rule update from a derived workflow and accepted finite contract, and proves that every finite continuation preserves Lean’s `AgentSec`. `fresh_alias_installation_serialized` proves both possible orders between a protected call and that update. `compilation_preload_preserves_agentSec` models preparation as an explicit transition and proves that preparing an inactive candidate preserves the same invariant. The appendix proves the general pomset lift, source-outcome preservation maps, information lower bound, and ideal-runtime weak bisimulation in sections A-A and A-L.

All 19 paper-specific tests pass, together with 62 authorization and checker regression tests for supporting components. Across accepted instances with 2–128 outcomes, checker time is 0.11–53.47 ms. Rejected instances take 0.11–5.51 ms, and enumerating 6–720 unordered completions takes 0.33–50.35 ms. Section A-M reports the complete measurement protocol and executable coverage.

VIII. RELATED WORK

a) Recovery, workflow update, and synthesis: Output-commit recovery reconstructs a fixed computation. Generalized nonblocking and Live Synthesis solve supplied transition systems and completion conditions, while workflow inheritance and dynamic controller update connect supplied old and new specifications [21], [22], [16], [17], [18], [23], [24], [25]. Our controller derives the edited workflow, action reuse, still-required outcomes, completion conditions, and new runtime rules directly from the execution record. It computes every

safe continuation and puts the corresponding rules into effect atomically with new authorization or authorization reuse.

b) Agent recovery mechanisms: ACRFence blocks replay after Restore, DART selects checkpoints, Atomix delays external actions until commit, and Rebound makes rollback atomic and auditable [7], [15], [13], [26]. Our theorem gives exact criteria for Checkpoint and all six derived Fork, Restore, and Merge forms, a rejection proof, and a lower bound proving that all four parts of the execution record are necessary. It carries these results through the atomic rule update and arbitrary finite execution.

IX. CONCLUSION

Safe execution editing has an exact criterion. Past protected actions remain unchanged, and every still-required outcome retains a policy-safe completion. For Checkpoint and every registered Fork, Restore, Merge, or extension, our checker either verifies the operation directly, computes every safe continuation, or returns a finite proof that no safe runtime implementation exists. The Exact Safety Checking theorem connects this exact decision to the four necessary parts of the execution record, the atomic rule update, repeated execution edits, and equivalence with an ideal atomic machine. It provides a formal foundation for Agent runtimes that support Checkpoint, Fork, Restore, and Merge.

APPENDIX A
SUPPLEMENTARY PROOF APPENDIX

This appendix instantiates the proof objects used by theorem 2 over finite registered call models and atomic policy-domain rule versions. Independent domains retain the product interpretation stated in the paper.

A. The Declarative Definition and the Computed Result

Fix a well-formed safety-check instance $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$ for which the edit derivation yields $\delta = (\kappa_u, H_u, \mathcal{A}_u, \mathcal{C}_u, \eta^-, \eta^+)$. All sets in this subsection retain outcome labels, so two equal resolved words belonging to different outcomes remain different elements.

Definition 4 (Required source outcomes). Write $\text{kind}(u)$ for the constructor of u . Call either Fork constructor a Fork and either Restore constructor a Restore. For the matching source row in table I, define

u	$\text{SrcReg}(H, u)$
Fork	$\{(L, \mathcal{C}), (R, \mathcal{C})\}$
Restore	$\{(\text{cur}, \mathcal{C}), (k, \mathcal{C}_k)\}$
MergeSelect(g, w, σ)	$\{(w, \llbracket T_w \rrbracket), (\bar{w}, \llbracket T_{\bar{w}} \rrbracket)\}$
MergeJoin(g, σ)	$\{(L, \llbracket T_L \rrbracket), (R, \llbracket T_R \rrbracket)\}$

The full set of source outcomes is

$$\begin{aligned} \text{RequiredBefore}(H, u) = & \{ \text{ctx} \} \times O_{\text{unchanged}} \\ & \uplus \biguplus_{\substack{(r, \mathcal{D}) \\ \in \text{SrcReg}(H, u)}} (\{r\} \times O_{\mathcal{D}}). \end{aligned}$$

Write $\text{Done}_H(r, o)$ when the recorded progress for region r shows that every event in outcome o 's pomset has completed. Only RestoreReplace treats completed current outcomes as finished:

$$\text{DoneCur}_H = \{(\text{cur}, o) \mid o \in O_{\mathcal{C}} \wedge \text{Done}_H(\text{cur}, o)\},$$

u	$\text{Satisfied}_{H, u}$
RestoreReplace(b, k, σ)	DoneCur_H
otherwise	$\emptyset$

Completion observability makes this current-role set either all of $\{\text{cur}\} \times O_{\mathcal{C}}$ or empty. The only sets eligible for signed removal are

$$\text{CurOut} = \{\text{cur}\} \times O_{\mathcal{C}}, \quad \text{Other}_w = \{\bar{w}\} \times O_{\llbracket T_{\bar{w}} \rrbracket},$$

u	$\text{CanRemove}(H, u)$
RestoreReplace(b, k, σ)	$\text{CurOut} \setminus \text{Satisfied}_{H, u}$
MergeSelect(g, w, σ)	Other_w
otherwise	$\emptyset$

$\text{reqauth}_H(a)$ follows a 's source region to the unique signed authority record in G , or to the stored copy in K when a comes from a checkpoint. Context identity, carry, and clone preserve that record, while every source region introduced by an edit rule receives the authority that signed the rule and policy version. Thus reqauth_H is total and immutable on $\text{RequiredBefore}(H, u)$, including Fork-created L/R copies.

For every $a \in \text{CanRemove}(H, u)$, the removal request must contain a signature by $\text{reqauth}_H(a)$ over

$$\begin{aligned} \text{rmmsg}(H, u) = & (\omega, \kappa, \zeta, \sigma, H, \\ & \text{CanRemove}(H, u), \\ & \text{origin}(\text{CanRemove}(H, u))). \end{aligned}$$

Without all required signatures, the edit has no derivation. With them, $\text{AuthorizedRemoval}_{H, u} = \text{CanRemove}(H, u)$. Write $\text{Sat}_u = \text{Satisfied}_{H, u}$ and $\text{Rm}_u = \text{AuthorizedRemoval}_{H, u}$. The checker requires

$$\begin{aligned} \text{StillRequired}_u = & \text{RequiredBefore}(H, u) \setminus (\text{Sat}_u \uplus \text{Rm}_u), \\ \text{RequiredBefore}(H, u) = & \text{StillRequired}_u \uplus \text{Sat}_u \uplus \text{Rm}_u. \end{aligned}$$

Definition 5 (Required-outcome preservation).

$$\begin{aligned} \text{PreservesRequired}(H, u, \hat{B}) \iff & \forall a \in \text{StillRequired}_u. \\ & \exists (t, y) \in \hat{B}. \text{Covers}_u(t, a). \end{aligned}$$

Here $\text{AuthorizedRemoval}_{H, u}$ contains the typed unselected-arm outcomes for MergeSelect or the current-role outcomes of an authorized RestoreReplace. $\text{Satisfied}_{H, u}$ contains the completed current role of RestoreReplace. Both sets are empty for every other row and for a registered extension. Thus $\text{StillRequired}_u = \text{RequiredBefore}(H, u) \setminus (\text{Satisfied}_{H, u} \uplus \text{AuthorizedRemoval}_{H, u})$, so PreservesRequired covers exactly the source outcomes that are neither completed nor authorized for removal.

Let $\mathfrak{R}_\Theta$ be the finite set of safe execution sets from definition 2, ordered componentwise by

$$(L_1, \hat{B}_1) \sqsubseteq (L_2, \hat{B}_2) \iff L_1 \subseteq L_2 \wedge \hat{B}_1 \subseteq \hat{B}_2.$$

Lemma 8 (The computed result is exact). The declarative predicate $\text{MaxSafe}(\Theta; L, \hat{B})$ holds for some pair if and only if $\hat{B}_{H, u}^\dagger \neq \emptyset$. When such a pair exists, it is unique and equals $(W_{H, u}^\dagger, \hat{B}_{H, u}^\dagger)$. This equality is a theorem relating two independently stated objects, rather than the definition of the ideal machine.

Proof. Let $(L, \hat{B}) \in \mathfrak{R}_\Theta$. The workflow, outcome, and policy requirements give $\hat{B} \subseteq \hat{B}_0$. For every $(o, y) \in \hat{B}$, every prefix $z \preceq_{\text{pref}} y$ belongs to $L = \text{Pref}(\pi_2 \hat{B})$. Persistent outcome coverage therefore supplies, for every $o' \in \text{Compat}(z)$, a pair $(o', y') \in \hat{B}$ extending z . Thus $\hat{B} \subseteq \hat{\Phi}(\hat{B})$. By monotonicity on the finite lattice $2^{\hat{B}_0}$, every post-fixed set is contained in the greatest fixed point, so $\hat{B} \subseteq \hat{B}^\dagger$. Taking projected prefixes yields $L \subseteq W^\dagger$.

Conversely, suppose $\hat{B}^\dagger \neq \emptyset$. The fixed-point equality keeps every compatible outcome completable and, at ϵ , combines with the checked preservation maps to establish PreservesRequired. Inclusion in $\hat{B}_0$ ensures that complete executions follow the target workflow and policy. Prefix closure of $\mathcal{A}_u$ makes every generated prefix safe, and $W^\dagger = \text{Pref}(\pi_2 \hat{B}^\dagger)$ gives a completion after every prefix. Hence $(W^\dagger, \hat{B}^\dagger) \in \mathfrak{R}_\Theta$, and the preceding containment makes it greatest. If two greatest elements existed, each would contain the other componentwise, so they would be equal. If $\hat{B}^\dagger = \emptyset$, the first half places

every hypothetical safe execution set inside the empty set, contradicting its required nonemptiness. $\square$

Corollary 1 (Exact checking). *The ideal edit transition is enabled by the declarative SafeBehavior(Θ) exactly when the checker returns a nonempty fixed point. On that edge, the ideal language L^* equals the generated language of $\text{Can}(W^\dagger, B^\dagger)$.*

Proof. The first statement is lemma 8. The second combines that lemma with lemma 6. $\square$

Proposition 2 (Output-sensitive checking). *Fix the unit-cost symbolic-checker model with constant-time finite-map lookup and policy-automaton transitions. Let*

$$\begin{aligned} D &= |\Theta|_{\text{chk}}, & n &= \max_o |X_{p_o}|, \\ \Lambda &= \sum_o \sum_{w \in \text{Lin}(p_o)} (|w| + 1), & N_B &= |\widehat{B}_0|, \\ V_B &= \sum_{(o,y) \in \widehat{B}_0} (|y| + 1). \\ \mathcal{G}_{\text{cov}} &= \left\{ ((o,y), z, o') \mid \begin{array}{l} (o,y) \in \widehat{B}_0, \ z \preceq y, \\ o' \in \text{Compat}(z) \end{array} \right\}. \end{aligned}$$

Here D is the checked input size, Λ is the total size of the emitted linearizations, and $\mathcal{G}_{\text{cov}}$ contains the candidate, prefix, and required-outcome triples examined by the checker. The checker runs in $O(D + n\Lambda + |\mathcal{G}_{\text{cov}}| + V_B \log(2 + V_B))$ time and $O(D + \Lambda + |\mathcal{G}_{\text{cov}}|)$ space, including the data it returns. Moreover, $N_B \leq \sum_o |\text{Lin}(p_o)|$, $|\mathcal{G}_{\text{cov}}| \leq N_B(n+1)|O_{C_u}|$, and the number of nonempty strict-removal ranks is at most N_B .

Proof. A backtracking topological-order enumerator emits all indexed linearizations in $O(n\Lambda)$ time, while resolution, policy simulation, and construction of raw and resolved prefix tries are linear in emitted volume. For every support key (z, o') , maintain the number of remaining pairs (o', y') extending z , initially enqueue candidates having a required outcome with zero support, and remove candidates in synchronous batches. When removal makes a support count zero, enqueue exactly the candidates that depend on that requirement for the next batch. Each candidate and each triple in $\mathcal{G}_{\text{cov}}$ is processed at most once, and the batches are exactly $\widehat{B}_i \setminus \widehat{B}_{i+1}$. Store one rank and cause per removed candidate rather than copied sets, and construct $\text{Can}(W, B)$ by bottom-up sorting of finite-trie derivative signatures in $O(V_B \log(2 + V_B))$ time. $\square$

B. Ordered Rejection Proofs

For an empty fixed point, the returned proof is

$$C_{\text{fp}} = (\widehat{B}_0, E_0, \dots, E_k), \quad E_i \subseteq \widehat{B}_i \times \text{Pref}(\pi_2 \widehat{B}_i) \times O_{H,u}.$$

The verifier first recomputes the canonical $\widehat{B}_0(\Theta)$ and requires equality with the serialized first component. An entry $((o,y), z, o') \in E_i$ verifies exactly when $z \preceq y$, $o' \in \text{Compat}(z)$, and no $(o', y') \in \widehat{B}_i$ extends z . The verifier requires the set of first components in E_i to equal $\widehat{B}_i \setminus \widehat{\Phi}(\widehat{B}_i)$, sets $\widehat{B}_{i+1} = \widehat{B}_i \setminus \pi_1(E_i)$, and accepts only when $\widehat{B}_{k+1} = \emptyset$.

All sets, prefixes, and compatibility checks are finite and use the canonical order fixed by the checker.

Lemma 9 (The rejection proof is sound and complete). *The verifier accepts C_{fp} iff the descending fixed-point computation reaches $\widehat{B}^\dagger = \emptyset$. In that case no safe execution set exists.*

Proof. The initial equality check ties the proof to Θ , and every verified round is exactly $\widehat{B}_{i+1} = \widehat{\Phi}(\widehat{B}_i)$, so canonical iteration supplies completeness. For soundness, induction places every post-fixed family inside every $\widehat{B}_i$, so an empty final family excludes every nonempty safe execution set. $\square$

C. Exact Returned Proof Data

Let enc be canonical and length-delimited, with κ authenticating $\mathcal{A}$. The symbolic LTS uses domain-separated injective authenticated digests $\text{Digest}_{\text{rules}}$ and $\text{Digest}_{\text{record}}$, so equal digests have equal encoded inputs. For $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, define

$$\gamma(\Theta) = \text{Digest}_{\text{record}}(\text{enc}(\text{request}, \omega, \zeta, \kappa, H, \Delta, \text{out}, \mathcal{A}, u)).$$

When $\text{Derive}(\Theta)$ is defined and $\widehat{B}_{H,u}^\dagger \neq \emptyset$, also define

$$\begin{aligned} \text{aut}_Z &= \text{Can}(W_{H,u}^\dagger, B_{H,u}^\dagger), \\ H_{\text{aut}} &= \text{Digest}_{\text{rules}}(\text{enc}(\text{aut}_Z, \widehat{B}_{H,u}^\dagger)). \end{aligned}$$

The digest checked before the rules change is

$$\text{cut}_Z = \text{Digest}_{\text{record}}(\text{enc}(\omega, \zeta, \kappa, H, \Delta, \text{out}, u, C_{H,u}, H_{\text{aut}}, \eta^-, \eta^+)). \quad (11)$$

Because H contains $\chi, \text{ver}, \rho, \beta$, the digest covers workflow progress, record version, registry contents, active rule entries and versions, the authorization log, and the request queue. The verifier re-derives the edit-rule judgment and complete fixed point labeled by outcome, reconstructs $(W^\dagger, B^\dagger)$, verifies both digests, and checks the full automaton isomorphism

$$\text{aut}_Z \cong \text{Can}(W^\dagger, B^\dagger),$$

including its initial state, marking, and every labeled transition. Canonical encoding order makes Invalid, Reject, and Accepted unique and unchanged by consistent renaming of internal identifiers. A byte implementation may authenticate enc with signatures or collision-resistant digests. The byte-level guarantee is parameterized by signature unforgeability and digest collision resistance, represented as authenticated identities in the formal model.

D. Proofs for Edit Rules and Rechecking

1) Edit-rule correctness:

Full proof of lemma 2. Unique object and edit-rule identifiers select one row and record, while context identity preserves everything outside the edited region. The Fork rows check both copies. RestoreReplace checks the checkpoint copy and the exact treatment of the current continuation, while RestoreLive checks both the retained current continuation and the checkpoint copy. MergeSelect checks the selected and unselected regions, while MergeJoin checks both retained regions before its barrier.

Clone and carry supply the global bijections and preserve which workflow calls belong to each outcome and which authority controls each source region. Constructor induction defines pr_r^u , proves the cover, and supplies every Covers_u witness. The common derived record fixes all remaining fields, preserves or initializes branch progress, and assigns each source region introduced by an edit rule its signed authority record. Fresh identifiers, partial-constructor checks, and lemma 1 preserve uniqueness, acyclicity, and contract well-formedness. Finally, $\beta'(x) = (j_x^+, \eta^+)$ and $(x, j_x^+, h_x^+) \in \mathcal{H}_u^+$ for every current target workflow call x . $\square$

2) *Registered extension*: The Agent may name only an immutable record ξ that specifies the exact prior edit rules, registry, execution record, and policy, together with finite disjoint additions $D_\Sigma, D_\rho, D_\alpha$ and a successor policy $\mathcal{A}^+$. The set D_α contains the signed authority record for every newly introduced outcome and its source region. The policy authority and call authorization service verify ξ . Existing meanings remain unchanged, new identifiers are fresh and domain-scoped, and $\mathcal{A} \subseteq \mathcal{A}^+$. Let ζ^+, κ^+ , and $\mathcal{A}^+$ be the authenticated successor versions and policy named by ξ . For every $x \in X_{[T]}$, the controller deterministically creates j_x^+ and h_x^+ . For fresh η^+ , define

$$\begin{aligned} G^+ &= G \uplus D_\alpha, \\ \Sigma_{\zeta^+} &= \Sigma_\zeta \uplus D_\Sigma, \\ \rho^+ &= \rho \uplus D_\rho, \\ \mathcal{H}_\xi^+ &= \{(x, j_x^+, h_x^+) \mid x \in X_{[T]}\}, \\ \beta^+(x) &= (j_x^+, \eta^+), \\ H_\xi^+ &= (\omega, G^+, T, K, \Sigma_{\zeta^+}, \rho^+, \beta^+, \\ &\quad \chi, \text{ver} + 1, \eta^+), \\ \mathcal{C}_\xi &= [T], \end{aligned}$$

$$\text{RequiredBefore}(H, \text{Extend}(\xi)) = \text{StillRequired}_{\text{Extend}(\xi)},$$

$$\text{StillRequired}_{\text{Extend}(\xi)} = \{\text{ctx}\} \times O_{[T]},$$

$$\text{Satisfied}_{H, \text{Extend}(\xi)} = \emptyset,$$

$$\text{AuthorizedRemoval}_{H, \text{Extend}(\xi)} = \emptyset,$$

$$\text{Covers}_{\text{Extend}(\xi)}(t, (\text{ctx}, o)) \iff t = o.$$

The checked judgment is

$$(\kappa, H, \mathcal{A}) \vdash \text{Extend}(\xi) \Downarrow (\kappa^+, H_\xi^+, \mathcal{A}^+, \mathcal{C}_\xi, \eta, \eta^+).$$

It preserves T, K, χ, Δ , out and every existing row of G, Σ_ζ, ρ , appends D_α to the execution record, and authorizes no removal. Every current outcome, workflow call, action, source region, authority record, and causal edge therefore remains unchanged. It uses the same checker and atomic rule update as an execution edit.

Lemma 10 (Registered extension preserves safety). *For a verified $\text{Extend}(\xi)$, the current safe executions after recorded progress form a post-fixed family for the extension instance and are therefore contained in $\widehat{B}_{H, \text{Extend}(\xi)}^\dagger \neq \emptyset$. Consequently, a valid registered extension cannot remove a currently required outcome or leave it without a safe completion.*

3) Rechecking after a prefix:

Full proof of lemma 5. Because z prefixes a projected member of $\widehat{B}^\dagger$, raw recoverability and streaming in lemma 3 give the recoverability and authorization-log equalities. Removing the executed prefix from the outcome-labeled executions retains every compatible outcome and gives

$$\text{Compat}_{\mathcal{C}/\text{raw}(z)}(s) = \text{Compat}_{\mathcal{C}}(zs).$$

Together with authorization-sequence concatenation, streaming resolution yields

$$\widehat{B}_0(\mathcal{C}, \Delta)/z = \widehat{B}_0(\mathcal{C}/\text{raw}(z), \Delta_z),$$

with the same retained outcome identities, policy, and target registry. Thus $\widehat{B}^\dagger/z$ is post-fixed for the residual operator and lies within its greatest fixed point Y . Conversely, streaming and residual policy safety put $zY = \{(o, zy) \mid (o, y) \in Y\}$ inside the original $\widehat{B}_0$. The union $\widehat{B}^\dagger \cup zY$ is post-fixed because prefixes $t \preceq z$ use existing witnesses, while prefixes $t = zs$ use witnesses in Y . Greatestness gives $zY \subseteq \widehat{B}^\dagger$, hence $Y = \widehat{B}^\dagger/z$. Finally, $\text{Pref}(B)/z = \text{Pref}(B/z)$ yields the generated-language equality. $\square$

Proof of lemma 10. The verified extension preserves the current workflow, which calls refer to the same action, every outcome label, and $\mathcal{A} \subseteq \mathcal{A}^+$. Every earlier safe completion and coverage witness therefore remains valid for the next fixed-point computation. The current residual family is therefore post-fixed, and greatestness gives its inclusion in a nonempty successor fixed point. $\square$

E. Lean Theorem Map

Six Lean modules machine-check the claims used in the paper. `FiniteCore` proves the executable greatest-fixed-point checker equivalent to the declarative safe-execution-set predicate, including streaming resolution and greatestness. `RegistrationRefinement` proves that registration accepts the typed finite model, every registered call remains under trusted control, action identity is preserved, and the finite and semantic contracts agree. `HistoryStructure` proves `deriveEdit_iff`, well-formedness preservation, and executable fixtures for Choice/Parallel Fork, Replace/Live Restore, Select/Join Merge. `OperationalSemantics` proves invariant preservation across every runtime trace, closure of all six edits, registered extension safety, and both orders between a protected call and an atomic rule update. `CompilationBridge` constructs a secure atomic rule update from an accepted checker result and reflects every successful update back to that result. `PaperTheorems` combines these results into exact registered checking, six-edit derivation, preservation before rules take effect, trace safety, and serialization between calls and rule updates. The general pomset lift, information lower bounds, and ideal-runtime weak bisimulation complete the theorem in this appendix.

F. Why All Four Parts of the Record Are Necessary

We prove that every exact checker needs all four parts of the execution record. For each omitted part, we construct two valid records that become indistinguishable even though their correct answers are different.

Let $\mathcal{Q}_{\text{sec}}$ pair every valid checked record $V = \text{Sec}(S; c, r)$ with a well-typed request $q ::= \text{Decide}(u) \mid \text{TryUpdate}(u, Y)$. For such a record, let $\Theta_V(u) = \Theta_S(u)$ and $\text{Ready}_V(u, Y) = \text{Ready}(S, u, Y)$. For Decide , Ans returns $\text{Check}(\Theta_V(u))$. For TryUpdate , it returns the updated runtime state exactly when $\text{Ready}_V(u, Y)$ holds, and returns NoCommit otherwise.

Internal identifiers may be renamed consistently, while public identifiers remain fixed. We write $(V, q) \cong (V', q')$ when one type-preserving renaming maps the entire record and request to the other. An input function f is exact when indistinguishable reduced inputs always have the same correct answer up to such a renaming:

$$\begin{aligned} \text{Exact}(f) &\iff \forall (V_0, q_0) \in \mathcal{Q}_{\text{sec}}. \\ &\quad \forall (V_1, q_1) \in \mathcal{Q}_{\text{sec}}. \\ &\quad (f(V_0), q_0) \cong (f(V_1), q_1) \\ &\implies \text{Ans}(V_0, q_0) \cong \text{Ans}(V_1, q_1). \end{aligned}$$

The four parts of the record are as follows. **P** contains the workflow, checkpoints, edit rules, required outcomes, authorized removals, and the authority for each outcome. **I** records which workflow calls refer to the same protected action, together with each action's request, policy label, scope, source region, current rule entry, and authorization data. **E** records processed calls, current and saved progress, prior authorizations, retries, returned values, queued requests, and the current policy state. **C** records the policy version, edit-rule version, execution record version, and active rule version, together with the internal namespace and the digest checked before new rules take effect. Encodings, digests, authorization sequences, and returned proof data are computed from these four parts rather than treated as additional facts.

For $J \subseteq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, $\text{Keep}_J(V)$ retains the facts in J . It also removes every stored reference to an omitted fact and every computed value that depends on an omitted fact. This recursive deletion has a unique result because computed values depend acyclically on the four recorded parts. $\text{Drop}_{ca}(V)$ removes only the correspondence between workflow calls and protected actions, together with all data that could reconstruct it. $\text{Drop}_{ra}(V)$ similarly removes only the correspondence between prior authorization records and protected actions.

Lemma 11 (Omitting information is well defined). *Consistent renaming commutes with Keep_J , Drop_{ca} , and Drop_{ra} . Their results therefore do not depend on the spelling of internal identifiers or the byte encoding of computed data.*

Proof. A type-preserving renaming preserves references and dependencies. It therefore maps every deletion step to the corresponding step in the renamed record. Induction over the finite acyclic dependency order proves the result. $\square$

Omitted	Record pair	Correct answers
P	$q_{\mathbf{P}}^i = \text{Decide}(\text{MergeSelect}(g_i, L, \sigma));$ $T^0 = b_x : X \oplus_{g_0^{\text{open}}} b_y : Y,$ $T^1 = b_x : X \parallel_{g_1} b_y : Y.$	V^0 : safe; V^1 : Invalid because the mode is wrong.
I	$q_{\mathbf{I}} = \text{Decide}(\text{RestoreLive}(b_L, k, \sigma_{\mathbf{I}}))$ after safe empty-root/ForkChoice/Checkpoint; $\Delta = \epsilon, \mathcal{A} = \text{Pref}\{a\};$ $q_{\text{act}}^0(x) = q_{\text{act}}^0(y) = d_0;$ $q_{\text{act}}^1(x) = d_0 \neq d_1 = q_{\text{act}}^1(y);$ $\ell(d_0) = \ell(d_1) = a.$	$(X \otimes Y_k) \oplus Y$: one new, accept; two new reject <i>aa</i> .
E	$u_{\mathbf{E}} = \text{RestoreReplace}(b_L, k, \sigma_{\mathbf{E}}),$ $q_{\mathbf{E}} = \text{TryUpdate}(u_{\mathbf{E}}, Y_0); Y_0$ is checked at V^0 after safe empty-root/ForkChoice/Checkpoint/new; $V^0 \xrightarrow{\text{Dispatch}(d)} V^1$, changing only whether a queued request was released.	V^0 : rules take effect; V^1 : NoCommit because the checked record changed.
Drop_{ra}	$q_{-ar} =$ $\text{Decide}(\text{RestoreReplace}(b_L, k, \sigma_{-ar}))$ after safe empty-root/ForkChoice/Checkpoint/new; $q_{\text{act}}(x) = d, \mathcal{A} = \text{Pref}\{a\}$; same calls $\{c, x\}$, records $\{p\}$, and actions $\{d, e\}$; $(q_{\text{act}}(c), p) = (d, d)$ in V^0 , (e, e) in V^1 .	$\text{reuse}(x_k, d)$: accept; $\text{new}(x_k, d)$: reject <i>aa</i> .
C	$q_{\mathbf{C}} = \text{TryUpdate}(u_0, Y_0)$, with Y_0 computed at V^0 , is identical in both; the current rule versions and dependent digests differ.	V^0 : rules take effect; V^1 : NoCommit .

TABLE II

RECORD PAIRS THAT BECOME INDISTINGUISHABLE AFTER ONE FACT IS OMITTED BUT REQUIRE OPPOSITE ANSWERS.

Lemma 12 (Opposite answers require distinguishable records). *If $\text{Ans}(V_0, q_0) \not\cong \text{Ans}(V_1, q_1)$, then every exact f satisfies $(f(V_0), q_0) \not\cong (f(V_1), q_1)$.*

Proof. The result is the contrapositive of the definition of $\text{Exact}(f)$. $\square$

G. A Common Initial State and Five Record Pairs

Definition 6 (Checked registration). *A boundary-finite typed Agent workflow $\mathcal{W}$ is a finite-state outcome-labeled LTS whose complete paths end at outcome terminals and yield finitely many protected-call traces after internal τ -steps are erased. CheckReg rejects any reachable strongly connected component that can still reach an outcome and contains a protected-call edge, permits erased τ -cycles, and then enumerates the finite classes. $\mathcal{R}$ stores a candidate map λ , and $\text{BRuns}(\mathcal{R})$ is the finite outcome-indexed language obtained by linearizing its registered root pomsets and labeling every workflow call by its protected Use. $\mathcal{W} \sqsubseteq_{\lambda} \mathcal{R}$ means that λ gives a bijection from these traces to $\text{BRuns}(\mathcal{R})$ that preserves outcomes, call order, which calls refer to which protected actions, and the signed policy authority for each source region. For source executions e, e' , write $e \equiv_{\partial} e'$ when erasing internal τ -steps yields the same outcome-labeled protected-call trace. The check requires every protected action to have one certified canonical request and requires every reuse of that action to retain the same request. It also requires λ to be total, unique, and free of unregistered instructions, while preserving prior authorizations and every required outcome.*

Every pair below starts from a checked initial state and differs in one part of the execution record. After that part is omitted, the checker receives indistinguishable inputs even though the correct answers are opposite.

We construct the finite pairs from one empty initial state S_0 . A registered call model $\mathcal{R}$ contains finite contracts, edit rules, policy, the checked registration map, and authenticated entries for outcomes, workflow calls, protected actions, canonical requests, source regions, scopes, and labels. Before the root rules take effect, it contains no prior authorization, execution progress, queued request, rule version, returned value, or automaton state. The registration relation has a step $S_0 \xrightarrow{\text{register}(\mathcal{W}, \mathcal{R}, n)} \text{Reg}(\mathcal{R}, n)$ exactly when $\text{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts and n is a fresh private namespace. A checked root update then initializes empty progress and request state, activates one rule version and the canonical automaton, and authenticates every current rule entry and root outcome authority.

Lemma 13 (Registration reflection). *For finite-state $\mathcal{W}$ and finite $\mathcal{R}$, $\text{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts iff the registered λ witnesses $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$ in definition 6. On acceptance, the λ -lowerings of source executions modulo $\equiv_\partial$ correspond bijectively to $\text{BRuns}(\mathcal{R})$. The correspondence preserves outcome identity, causal order, canonical requests, prior authorizations, and the safety-check answer.*

Proof. The SCC check terminates because the state graph is finite. A reachable SCC that can still reach an outcome and contains a protected-call edge can be repeated before that outcome, producing projected words of unbounded length. Conversely, if no such SCC contains a protected-call edge, contracting the SCCs yields a DAG with a finite, enumerable projected language. The checker compares that language with $\text{BRuns}(\mathcal{R})$ and checks outcomes, call order, call-to-action correspondence, protected-action authentication, total unique registration, exclusion of unregistered calls, and prior authorizations after every prefix. It therefore accepts exactly when $\mathcal{W} \sqsubseteq_\lambda \mathcal{R}$ holds. Erasing internal steps gives a bijection between matching protected-call traces. Canonical-request equality and deterministic Resolve preserve prior authorizations at every prefix. The safe execution sets are therefore order-isomorphic, so the fixed-point operator preserves the safety decision, greatestness, and corresponding automaton. $\square$

Lemma 14 (Checked initial state). *If $\text{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts and the registered root contract and policy have a nonempty greatest safe execution set, the checked root update reaches a finite state $S_{\mathcal{R}}$ with $\text{AgentSec}(S_{\mathcal{R}}; c_{\mathcal{R}}, \epsilon)$. The execution record stores the checked root derivation, and every later rule update stores its checked execution edit or registered extension.*

Proof. Registration contributes only the checked $\mathcal{R}$ and n , and deterministic allocation constructs every runtime identifier from them. The root judgment and checked lifts establish core/schema coherence, and the recomputed nonempty fixed point establishes required-outcome coherence. Empty runtime progress establishes execution coherence, the canonical au-

tomaton at q_ϵ establishes automaton coherence, and the atomic rule-entry assignments establish rule-version coherence. Only the initial rule update creates a root origin, and every successful later update stores its checked execution-edit or registered-extension derivation. $\square$

The following finite pairs use private names except for each displayed request and authorization label. Each state is reached from S_0 by the stated registration, initial rule update, Checkpoint, protected-call, and edit steps.

a) **P: workflow structure:** Let X and Y be singleton contracts with distinct workflow calls x and y , a shared protected action, and a universal residual policy. From the same registered singleton root, a checked initial rule update followed by ForkChoice reaches $V_{\mathbf{P}}^0$ with private group g_0 , while the corresponding ForkParallel trace reaches $V_{\mathbf{P}}^1$ with private group g_1 . The two records differ only in

$$V_{\mathbf{P}}^0.T = b_L:X \oplus_{g_0}^{\text{open}} b_R:Y, \quad V_{\mathbf{P}}^1.T = b_L:X \parallel_{g_1} b_R:Y.$$

Use the same registered MergeSelect schema with $q_{\mathbf{P}}^i = \text{Decide}(\text{MergeSelect}(g_i, L, \sigma))$. The choice record has a unique edit derivation and a nonempty one-step safe execution set. The parallel record has no MergeSelect derivation because its group mode is wrong. Omitting **P** removes the constructor tag and all computed authentication data that depends on it. The private renaming $g_0 \mapsto g_1$ then makes the remaining record-request pairs indistinguishable.

b) **Common start for the correspondence pairs:** Let **1** be the singleton-outcome contract whose unique pomset is empty. Its completion ϵ gives a nonempty greatest safe execution set under $\mathcal{A} = \text{Pref}\{a\}$. Both pairs below put these safe root rules into effect and then apply a checked ForkChoice. Any inactive edit-rule entry needed to equalize the individual protected actions is identical in both records and never becomes active.

c) **I: which calls refer to the same action:** Let X, Y be singleton contracts with workflow calls x, y , and let d_0, d_1 be protected actions without authorization records, both labeled a . Two checked call models contain the same workflow and the same individual calls and actions, but the calls refer to the actions differently:

	$q_{\text{act}}(x)$	$q_{\text{act}}(y)$
$V_{\mathbf{I}}^0$	d_0	d_0
$V_{\mathbf{I}}^1$	d_0	d_1

From the **1** root now in force, the common Fork rule derives $b_L:X \oplus_g^{\text{open}} b_R:Y$. Each outcome uses one protected action, so both Fork updates are safe under $\text{Pref}\{a\}$. Create a checkpoint for b_R before progress and issue the common query $q_{\mathbf{I}} = \text{Decide}(\text{RestoreLive}(b_L, k, \sigma_{\mathbf{I}}))$. Up to clone renaming, the target is $(X \otimes Y_k) \oplus Y$. In $V_{\mathbf{I}}^0$, the left outcome uses one new and one reuse on d_0 , while the bypass outcome uses one new, so every outcome has authorization sequence a . In $V_{\mathbf{I}}^1$, every left-outcome linearization uses new on both d_0, d_1 , so its word is $aa \notin \mathcal{A}$. The left outcome is compatible with ϵ , so the first Φ round also removes the otherwise safe bypass completion and leaves the greatest fixed point empty. Applying

Drop_{ca} makes the remaining records indistinguishable because their individual calls and actions were already identical.

d) **E: authorization and execution progress:** Let C, X be singleton contracts with workflow calls c, x , let $q_{\text{act}}(c) = q_{\text{act}}(x) = d$, and label d by a . From the safe **1** root, a checked Fork update puts $b_L:C \oplus_g^{\text{open}} b_R:X$ into effect. Create a checkpoint for b_R , then use c to complete the left arm, create authorization record p , and queue its protected request. At the resulting V_E^0 , check $u_E = \text{RestoreReplace}(b_L, k, \sigma_E)$ to obtain Y_0 . Its clone of x reuses d , so the update is safe under $\mathcal{A} = \text{Pref}\{a\}$. Let $V_E^0 \xrightarrow{\text{Dispatch}(d)} V_E^1$, which changes only whether the queued request has been released and leaves **P, I, C** unchanged. For the same request $q_E = \text{TryUpdate}(u_E, Y_0)$, the rules take effect at V_E^0 , while V_E^1 returns NoCommit because the digest of the checked record has changed.

e) **Drop_{ra}:** which action a prior authorization names: For a separate pair, keep the common active row $q_{\text{act}}(x) = d$, creator c , authorization record p , and protected actions d, e labeled a , but set $q_{\text{act}}(c) = d$ and $p \mapsto d$ in $V_{\neg ar}^0$, versus $q_{\text{act}}(c) = e$ and $p \mapsto e$ in $V_{\neg ar}^1$. Both states follow the same safe empty-root/ForkChoice/Checkpoint/new shape above, have authorization sequence a , and use the common query $q_{\neg ar} = \text{Decide}(\text{RestoreReplace}(b_L, k, \sigma_{\neg ar}))$. The cloned x_k is reuse(x_k, d) in $V_{\neg ar}^0$, but new(x_k, d) produces forbidden word aa in $V_{\neg ar}^1$. The two records contain the same creator, authorization record, workflow call, and protected actions, and both retain $x \mapsto d$. Drop_{ra} removes the only differing correspondence and every value that could reconstruct it, so the remaining records are indistinguishable. This pair proves that an exact checker must know which protected action each prior authorization names.

f) **C: the active rule version:** Let a bootstrap namespace be a family $n = (n_s)_{s \in \text{Sort}}$ of injective, per-sort fresh supplies. From the same empty initial state, register the same public call model $\mathcal{R}$ with supplies n^0, n^1 that agree on every type except the rule-version type. The checked root updates create distinct active versions $\eta_0 \neq \eta_1$. The resulting V_C^0, V_C^1 have identical **P, I, E**, while their active rule versions and dependent digests differ. Check the same registered edit u_0 at V_C^0 to obtain $Y_0 = \text{Accepted}(\delta_0, W_0, \hat{B}_0, Z_0)$, then submit $q_C = \text{TryUpdate}(u_0, Y_0)$ to both records. At V_C^0 , the final check reproduces cut _{Z_0} , so the rules take effect. At V_C^1 , the active version is η_1 rather than the η_0 recorded in Z_0 , so the update returns NoCommit. Omitting **C** also removes dependent rule entries, authorization data, and digests, leaving the record-request pairs indistinguishable.

Proposition 3 (Four necessary record parts). *Every pair above consists of finite runtime states reached from the common empty initial state and satisfying AgentSec. After the corresponding information is omitted, each pair becomes indistinguishable even though the correct answers are opposite. Therefore every Keep_J with $J \subsetneq \{\mathbf{P}, \mathbf{I}, \mathbf{E}, \mathbf{C}\}$, as well as Drop_{ca} and Drop_{ra}, is inexact. Every exact checker input must distinguish all five pairs.*

Proof. Lemma 14 gives the common initial state, and the traces

above use only modeled Checkpoint, protected-call, execution-edit, and rule-update steps. For the two correspondence pairs, the empty root completes with ϵ , and each Fork arm has a one-new completion labeled a . Hence every displayed state before its request is reachable under $\text{Pref}\{a\}$. Open choice enables the Checkpoint steps, and the active automata enable each stated left-arm new before that arm becomes a completed, still-addressable continuation. For the **E** pair, Dispatch preserves **P, I, C** but changes the checked record, giving a successful update versus NoCommit. The displayed edit-rule and resolution calculations give the other opposite answers. Lemma 11 and the stated private renamings make the reduced inputs indistinguishable. Each omission collapses its corresponding pair, so lemma 12 proves inexactness. $\square$

Giving the **P** pair the same caller-proposed target system and completion conditions leaves their opposite answers unchanged. The target alone is therefore insufficient. A generalized nonblocking solver becomes applicable only after the trusted controller has derived the edited workflow, preserved outcome identities, and outcome-specific completion conditions.

H. Ideal Atomic Execution Machine

An ideal state $I = (\kappa, H, \Delta, \text{out}, \mathcal{A}, c, r)$, with $c = (H_c, u, L^*, \hat{B}^*)$, contains the execution record, current rule version and entries, checked safe execution set, and current partial execution. Write $\Theta_I(u) = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$. The runtime state additionally represents the automaton, transaction phase, authentication data, and a prepared rule update.

a) **Edit transition:** For current $\Theta_I(u)$, an undefined derivation gives state-preserving Invalid, while a defined derivation with no MaxSafe witness gives state-preserving Reject. Otherwise $\text{Derive}(\Theta_I(u))$ supplies the target and rule version, and independently defined $\text{SafeBehavior}(\Theta_I(u)) = (L^*, \hat{B}^*)$ enables one atomic $\text{Edge}(u, [\mathcal{H}_u^+]_{\cong})$ that makes them current and resets r to ϵ . All three answers are computed from the same current execution record and are mutually exclusive and exhaustive.

b) **Protected call:** For submitted tool request m , an enabled x passes the ideal automaton exactly when its current rule entry and authorization data identify protected action d , $\text{canon}_{\kappa}(m) = \text{inv}_{\rho}(d)$, and $ra \in L^*$. Here $a = \text{new}(x, d)$ if d has no authorization record, and $a = \text{reuse}(x, d)$ otherwise. One atomic transition applies HStep, appends a to χ , advances ver, removes x 's current rule entry, and sets $r := ra$. new also creates the labeled authorization record and queues the canonical request, while reuse links to the existing return record. Any failed premise returns $\text{deny}(x)$ without changing state or inventing a return value.

c) **Checkpoint, retry, release, and restart:** Checkpoint, authenticated retry, Dispatch, completion updates, and restart after a stop have the state changes in table III. This least-generated LTS represents post-Dispatch external-network completion as environmental behavior.

By lemma 4, every reachable partial execution of the ideal machine retains a policy-safe completion until the next edit

transition changes the workflow contract. The runtime implements this machine without enumerating the full completion set at every call.

I. Runtime State and Atomic Transitions

For $\Theta = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, let $C_{\text{der}}(\Theta)$ be the canonical failed-derivation transcript.

$$\begin{aligned} \text{Check}(\Theta) &= \text{Invalid}([\gamma(\Theta), C_{\text{der}}(\Theta)]_{\cong}), \\ &\quad \text{if } \text{Derive}(\Theta) \text{ is undefined;} \\ \text{Check}(\Theta) &= \text{Reject}([\gamma(\Theta), C_{\text{fp}}]_{\cong}), \\ &\quad \text{if } \text{Derive}(\Theta) \text{ is defined and } \hat{B}_{H,u}^{\dagger} = \emptyset. \end{aligned}$$

When $\text{Derive}(\Theta)$ is defined and $\hat{B}_{H,u}^{\dagger} \neq \emptyset$,

$$\text{Check}(\Theta) = \text{Accepted}(\delta, W_{H,u}^{\dagger}, \hat{B}_{H,u}^{\dagger}, Z).$$

where

$$Z = (\omega, \zeta, \kappa_Z, \text{cut}_Z, u, H_{\text{aut}}, \eta^-, \eta^+).$$

Invalid carries the failed derivation, Reject carries the complete ordered rejection proof, and Accepted carries the unique derivation, coverage proof, canonical automaton, and Z . Z authenticates the checked execution record, derived target, automaton, and both old and new rule versions.

The runtime state for one policy domain is

$$S = (\kappa, H, \Delta, \text{out}, \mathcal{A}, \text{phase}, \text{entry}, \text{aut}, q, \text{checked}),$$

where phase records whether each rule version η is unallocated ($\perp$), inactive, active, or closed. The bootstrap rule runs only after $\text{CheckReg}(\mathcal{W}, \mathcal{R})$ accepts, and its exact construction appears in section A-G. Each active version stores $c = (\text{record}_c, H_c^+, u, W, \hat{B}, Z)$, which contains the checked execution record, derived target, edit, safe prefixes, outcome-labeled complete executions, and data needed for the final recheck.

Definition 7 (Agent runtime invariant). For $c = (\text{record}_c, H_c^+, u, W, \hat{B}, Z)$, $\text{record}_c = (\kappa_c, H_c, \Delta_c, \text{out}_c, \mathcal{A}_c)$, and resolved r , $\text{AgentSec}(S; c, r)$ holds exactly when the following clauses hold.

- 1) Record and edit-rule coherence. The state is well formed, $c = \text{checked}(H, \eta)$, its origin derives the target now in force, and every required outcome has one authenticated authority and a checked Covers_u proof.
- 2) Required-outcome coherence. $\hat{B} = \hat{B}_{H_c, u}^{\dagger} \neq \emptyset$, $W = \text{Pref}(\pi_2 \hat{B})$, and Z verifies both against record_c .
- 3) Execution coherence. $(H.G, H.T) = \text{HRes}((H_c^+.G, H_c^+.T), r)$, $H.\chi = H_c^+.\chi r$, and authorization records, queue entries, return links, and checkpoints match r and advance only monotonically.
- 4) Automaton coherence. $\text{aut}(H, \eta) \cong \text{Can}(W, \pi_2 \hat{B})$, and $q(H, \eta)$ is its derivative after r .
- 5) Rule-version coherence. Only H, η is active, $\text{entry}|_{X[\mathbb{H}].T} = H.\beta$, and it provides exactly one authenticated rule entry for each current workflow call, while inactive or closed versions provide none.

a) *Atomic protected call*: For submitted tool request m , set $m^\circ = \text{canon}_{\kappa}(m)$. For a call x with current rule entry j in version η , $\text{VerifyToken}(h, \rho(x), j, d, m^\circ)$ verifies the call authorization service, scope, rule entry, protected action, source region, and signed digest and requires $m^\circ = \text{inv}_{\rho}(d)$. Set $a = \text{new}(x, d)$ if $d \notin D_{\Delta}$, otherwise $a = \text{reuse}(x, d)$, and let $\text{Use}_x(m) = \text{Use}(x, j, h, m)$. Its sole state-changing transition is

$$\frac{\begin{array}{l} x \text{ enabled in } T \quad \text{entry}(x) = (j, \eta) \\ \eta = H, \eta \quad \text{phase}(\eta) = \text{active} \\ \text{VerifyToken}(h, \rho(x), j, d, m^\circ) \quad q(\eta) \xrightarrow{a}_{\text{aut}(\eta)} q' \end{array}}{S \xrightarrow{\text{Use}_x(m)/a} S'} \quad (12)$$

The paired update $\text{HStep}((G, T), a)$ removes x , records any choice, preserves the other workflow constructors, and appends a to its branch progress. $\text{Advance}(S, x, a, q')$ changes exactly

$$\begin{aligned} (H.G, H.T) &:= \text{HStep}((G, T), a), & H.\chi &:= \chi a, \\ H.\text{ver} &:= \text{ver} + 1, & H.\beta &:= \beta \setminus \{x\}, \\ \text{entry} &:= \text{entry} \setminus \{x\}, & q(\eta) &:= q'. \end{aligned}$$

In addition, new appends the immutable labeled authorization record and queues the canonical request $(d, \text{inv}_{\rho}(d))$, never an unchecked caller request. reuse records the protected action's stored return link without changing Δ , and all other fields remain unchanged. An authenticated Retry follows the archived link from χ to the authorization record for the same canonical request and returns any stable value without changing state.

b) *Final check and atomic rule update*: At the atomic update point, the trusted controller ignores any caller-supplied replacement workflow or rule version, sets $\Theta_S(u) = (\kappa, H, \Delta, \text{out}, \mathcal{A}, u)$, and re-runs $\text{Derive}(\Theta_S(u))$ from the current execution record. For a positive derivation, $\text{ReCut}(S, u, Z)$ recomputes the right-hand side of equation (11), while Verify_Z recomputes the fixed point, coverage proof, canonical automaton, and both digests. The rules may take effect only if

$$\begin{aligned} \kappa &= \kappa_Z, & \text{ReCut}(S, u, Z) &= \text{cut}_Z, \\ \text{Verify}_Z(W_{H,u}^{\dagger}, B_{H,u}^{\dagger}, \hat{B}_{H,u}^{\dagger}), & & & \\ H, \eta &= \eta^-, & \text{phase}(\eta^+) &\in \{\perp, \text{inactive}\}. \end{aligned} \quad (13)$$

$\text{Ready}(S, u, Y)$ holds when $Y = \text{Accepted}(\delta, W^{\dagger}, \hat{B}^{\dagger}, Z)$, the current derivation is δ , and all state-dependent premises of equation (13) hold. The committing domain transaction makes $(\kappa_u, H_u, \mathcal{A}_u)$ current, closes η^- , activates η^+ , assigns a rule entry to every target workflow call, starts the canonical automaton at q^0 , and stores the checked result. For this successor, write

$$c_{\delta}^+ = (\text{state}(\Theta_S(u)), H_u, u, W_{H,u}^{\dagger}, \hat{B}_{H,u}^{\dagger}, Z).$$

Only after activation does it expose $\mathcal{H}_u^+$ and emit $\text{Edge}(u, [\mathcal{H}_u^+]_{\cong})$. Old authorization data then fails, while Retry still follows its archived authorization-record link. A stale result returns NoCommit and must be recomputed from the current execution record.

J. Atomic Events Preserve the Invariant

Write $A_1, \dots, A_5$ for the five clauses of AgentSec in their stated order. The following matrix covers every state-changing transition after the initial rules take effect. Initial registration occurs before the root safety check, and later additions use the registered-extension transition below.

Lemma 15 (Atomic events are well defined). *The atomic update induces a partial function ∂ on runtime-state and event pairs modulo consistent renaming, written $\langle V, e \rangle_{\cong}$. If $\text{AgentSec}(S; c, r)$ and $S \xrightarrow{e} S'$, the witness in table III satisfies $\text{AgentSec}(S'; c', r')$ and*

$$[\text{Sec}(S'; c', r')]_{\cong} = \partial(\langle \text{Sec}(S; c, r), e \rangle_{\cong}).$$

Proof. All transition rules use typed equality, membership, order, canonical constructors, and authenticated-data equality. These operations are unchanged by consistently renaming private identifiers in the state and event. For an event e , let $G_e(V)$ be its precondition and $U_e(V)$ its successor update when that precondition holds. For every such renaming π , $G_e(V) \iff G_{\pi e}(\pi V)$ and $U_{\pi e}(\pi V) = \pi U_e(V)$. Thus consistently renamed inputs have the same definedness and consistently renamed successors. This proves that ∂ is well defined.

For new and reuse, lemmas 3, 5 and 6 establish the first two rows of the matrix. If x lies at b , HStep appends a to b 's progress, while the same atomic protected-call update appends a to the global resolved-call trace χ . Associativity of language quotients gives

$$(\Gamma_b / \text{raw}(\chi_b)) / x = \Gamma_b / \text{raw}(\chi_b a).$$

Thus the updated leaf and any later Checkpoint remain synchronized with G . For a successful edit transition, lemmas 2, 6 to 8 and 10 establish the third row for all six edit rules and registered extension. Each remaining rule changes only the fields shown in the matrix, and its row checks every invariant clause whose fields change. Thus every successor has the displayed new state and preserves AgentSec. $\square$

Corollary 2 (Finite-prefix closure). *Starting from any valid bootstrap, every finite prefix of any sequence of the event classes in table III ends in a state satisfying AgentSec. The sequence length is unbounded, although every individual safety-check instance and event payload is finite. For a finite word $\bar{e}$, the iterated update is defined on $\langle V, \bar{e} \rangle_{\cong}$, where one renaming acts on the initial state and the whole word.*

Proof. The base case is lemma 14, and lemma 15 supplies the invariant-preserving inductive step and consistency under renaming the remaining event suffix. $\square$

K. Events Outside the Agent's Control and Restarts

The resolved alphabet contains only protected calls checked by the runtime automaton. Agent request arrival, scheduling, completion, delivery, and stop or restart events do not append a new or reuse symbol. Request arrival and scheduler choice do not change committed runtime state. Completion updates and

delivery emit modeled API observations but leave the checked partial execution unchanged. Dispatch is observable only when the oldest queued request is recorded as released, and it also leaves the resolved prefix unchanged. Therefore none of these events advances the automaton for protected calls.

Lemma 16 (Other events preserve the checked prefix). *If $\text{AgentSec}(S; c, r)$, e is one of the modeled events above, and $S \xrightarrow{e} S'$, then the resolved prefix and canonical automaton state are unchanged and $\text{AgentSec}(S'; c, r)$ holds.*

Proof. Request arrival and scheduling do not change committed state, while denial, retry, retrieval, delivery, and restart preserve the checked runtime state. Request release and completion updates change only the queued-request status, authorization-record status, or stored return record permitted by execution coherence. None changes $H.T$, $H.\chi$, the authorization sequence derived from Δ , r , or $q(\eta)$, and the corresponding row of table III preserves every other invariant clause. $\square$

The restart theorem instantiates the storage interface of section III with linearizable atomic domain transactions. Let D be the committed runtime state, v the unfinished in-memory state, and $\bar{t} = t_1 \dots t_n$ the finite set of domain transactions overlapping a stop, ordered by their linearization points. Atomicity selects a prefix $\bar{t}_{\leq k}$ containing exactly the transactions that committed before the stop and gives

$$\text{Recover}(D, v, \bar{t}) = \text{commit}_{t_k} \circ \dots \circ \text{commit}_{t_1}(D),$$

with the empty prefix interpreted as D . No state reached after restart contains only part of one modeled atomic update. In particular, a new transaction cannot expose an authorization record without its global trace, branch progress, and queued request. A rule update cannot activate a new version without closing the old version and assigning every target rule entry.

Lemma 17 (Restart preserves an atomic prefix). *Let $S_0 \rightarrow S_1 \rightarrow \dots \rightarrow S_n$ be the modeled path whose atomic transitions correspond to $t_1, \dots, t_n$ in the order above. If committed state D refines S_0 , then restart after the committed prefix $t_1 \dots t_k$ yields a state refining S_k . At committed-state boundaries, stopping and restarting without another commit is consequently a τ self-loop.*

Proof. Induction on k starts from D refining S_0 . At each step, the complete atomic update commit_{t_i} is the update in the corresponding row of table III, so it maps a concrete state refining S_{i-1} to one refining S_i . The recovery equation applies exactly the first k updates and no partial update, yielding the claim. Once the LTS records the restarted state, stopping again changes only unfinished memory and is therefore a self-loop. $\square$

L. Agent-API Weak Bisimulation After Restart

We now prove theorem 5 by exhausting the observable and silent cases. For runtime state $S = (\kappa, H, \Delta, \text{out}, \mathcal{A}, \dots)$, define $\text{Core}(S) = (\kappa, H, \Delta, \text{out}, \mathcal{A})$ and $\text{Current}(H) = X_{[H.T]}$. For $c = (\text{record}_c, H_c^+, u, W, \hat{B}, Z)$, define $\text{icut}(c) =$

Event	Atomic change	New state	Why the invariant remains true
new use	Apply HStep to (G, T) ; append $\text{new}(x, d)$ to global χ ; advance ver , q ; append one authorization record and queued canonical request; remove the current rule entry.	$(c, r\text{new}(x, d))$	A_1 : record and edit rules unchanged. A_2 : rechecking after the prefix. A_3 : unique authorization record and request order. A_4 : canonical derivative. A_5 : remove exactly x 's rule entry.
reuse use	Apply HStep to (G, T) ; append $\text{reuse}(x, d)$ to global χ ; advance ver , q ; add only the return-record link; remove the current rule entry.	$(c, r\text{reuse}(x, d))$	A_1 : authenticated protected action retained. A_2 : rechecking after the prefix. A_3 : link names an earlier authorization record and the authorization sequence is unchanged. A_4 : canonical derivative. A_5 : remove exactly x 's rule entry.
Successful edit transition, six edits or registered extension	Make derived $(\kappa^+, H^+, \mathcal{A}^+)$ and the checked automaton current; close η^- ; activate η^+ ; publish new rule entries.	(c_δ^+, ϵ)	A_1 : edit-rule correctness and outcome preservation. A_2 : exact checker. A_3 : prior authorizations and queued requests retained. A_4 : start at q_e . A_5 : complete rule update.
Verified Invalid or Reject	Return the failed derivation or rejection proof; no runtime-state change.	(c, r)	All clauses unchanged; γ ties the answer to the checked execution record.
Checkpoint	Record the current continuation and progress in an authenticated checkpoint; advance ver .	(c, r)	A_1 : immutable well-typed extension. A_2, A_4, A_5 : unchanged. A_3 : exactly the permitted K extension.
Preload	Prepare content only in an inactive rule version assigned to no call.	(c, r)	A_1 – A_4 : unchanged. A_5 : the version remains inactive and unavailable to calls.
Retry, retrieval, delivery	Emit $\text{Return}(t, [v]_\cong)$ for an authenticated stored value; no runtime-state change.	(c, r)	All clauses unchanged; retry additionally checks the same canonical request and prior-authorization link.
Denial	Return $\text{deny}(x)$; no state mutation.	(c, r)	All clauses unchanged.
Dispatch(d)	Mark exactly the oldest queued canonical request as released.	(c, r)	A_3 : release order remains a prefix of queue order. Other clauses unchanged.
Completion update	Emit $\text{Settle}(d, [v]_\cong)$, advance one authorization record phase, and fill its stable return record.	(c, r)	A_3 : protected action, canonical request, creator, authorization label, and order are immutable. Other clauses unchanged.
Stale or malformed candidate	Return NoCommit; no runtime-state change.	(c, r)	All clauses unchanged.
Stop and restart	Discard unfinished in-memory work and expose the last committed runtime state.	(c, r)	All clauses unchanged because each atomic update appears either completely or not at all.

TABLE III
EXHAUSTIVE EVENT CLASSES AND PRESERVATION OF THE FIVE AgentSec CLAUSES.

$(H_c^+, u, W, \widehat{B})$. For $I = (K_I, c_I, r_I)$ with $K_I = (\kappa_I, H_I, \Delta_I, \text{out}_I, \mathcal{A}_I)$, let

$$\alpha_I(I) = (K_I, c_I, r_I, H_I.\eta, H_I.\beta),$$

$$\alpha_S(S; c, r) = (\text{Core}(S), \text{icut}(c), r, H.\eta, \text{entry}|_{\text{Current}(H)}).$$

Define IBS iff some c, r satisfy $\text{AgentSec}(S; c, r)$ and $\alpha_I(I) = \alpha_S(S; c, r)$; write $\xRightarrow{\ell} = \tau^* \ell \tau^*$ and $\xRightarrow{\tau} = \tau^*$. Fix IBS with witness (c, r) . Equality of $\alpha_I(I)$ and $\alpha_S(S; c, r)$ gives both machines the same κ , current policy, execution record, authorization log, request queue, stored return records, checked safe execution set, current partial execution, rule version, and current rule entries.

a) Edit transitions: The equality makes both machines form the same current $\Theta_I(u)$ and record digest $\gamma(\Theta_I(u))$. If derivation is undefined, both sides return the same canonical Invalid result and leave their states unchanged. If derivation exists but no safe execution set exists, lemma 8 and deterministic fixed-point checking make both sides return the same canonical Reject result and leave their states unchanged. Otherwise the ideal machine enables its edit transition and, by lemma 8, the runtime checker accepts the same edit and computes the same largest safe prefix set. The runtime takes an explicit Preload transition that records the prepared result in an inactive rule version, exposes no authorization data or Agent-visible result, and preserves AgentSec. The ideal side takes its single atomic edge, and both successors have the same $(\kappa_u, H_u, \mathcal{A}_u)$, checked safe execution set, empty prefix, active rule version, and target rule entries. Deterministic allocation gives consistently renamed authorization data, so both expose $\text{Edge}(u, [\mathcal{H}_u^+]_\cong)$. Conversely, every successful runtime rule update has passed the edit-rule, fixed-point, and record-digest checks, so lemma 8 enables the

matching ideal edge. This argument applies uniformly to all six execution edits and the registered extension.

b) Protected calls and submitted requests: For a submitted tool request m , both automata compute the same $m^\circ = \text{canon}_\kappa(m)$. The runtime authorization check verifies the call authorization service, scope, workflow call, rule entry, protected action, source region, signed digest, current rule version, and equality $m^\circ = \text{inv}_\rho(d)$. The ideal automaton requires the same rule entry, protected action, and request equality. Both sides then inspect the same set D_Δ of protected actions with authorization records, so they choose the same $a \in \{\text{new}(x, d), \text{reuse}(x, d)\}$. By corollary 1 and lemma 6, the runtime automaton enables a after r exactly when $ra \in L^*$ in the ideal machine.

In the new case, both successors apply the same workflow update, append the same symbol to the global trace and branch progress, advance ver , create the same authorization record, and queue the same canonical request. The runtime request queue stores $(d, \text{inv}_\rho(d))$, while the ideal queue stores (d, m°) . The verified request equality makes these entries identical. In the reuse case, both successors apply the same workflow update, append the same symbol to the global trace and branch progress, and link the workflow call to the same stored return record without changing Δ or the request queue. Both successors remove x 's current rule entry. Because the remaining workflow no longer contains x , their current-rule-entry components remain equal. In either case, lemma 15 re-establishes $\mathcal{B}$ with witness (c, ra) .

If any workflow call, rule entry, authorization datum, canonical request, rule version, branch, or automaton transition check fails, both automata fail. Both machines emit $\text{deny}(x)$, preserve state, and remain related.

c) *Races between a protected call and a rule update:*

Protected calls and rule updates serialize on the same execution record and active rule version. If a new use commits first, it changes $(G, T, \Delta, \chi, \text{ver}, \text{out})$, all of which are covered by the digest of the checked record, so the prepared rule update becomes stale. If an reuse use commits first, it changes (G, T, χ, ver) even though Δ and the request queue are unchanged, so the prepared rule update again becomes stale. If the rule update commits first, it atomically closes η^- , activates η^+ , refreshes every current rule entry and its authorization data, and exposes the new authorization data only after activation. The old new or reuse use then fails the current-rule-version automaton and is denied without progress. The ideal atomic order has exactly the same two cases, so neither side admits a third race outcome.

d) *Replies, releases, and silent steps:* An authenticated retry, authorization-data retrieval, or result delivery leaves the runtime state unchanged and follows the same authenticated link on both sides to expose $\text{Return}(t, [v]_{\cong})$. $\text{Dispatch}(d)$ is enabled for the same oldest queued request and makes the same observable release update. $\text{Settle}(d, [v]_{\cong})$ is enabled for the same authorization record and produces the same stable result and normalized update. Checkpoint takes matching τ -steps. Runtime preload and internal NoCommit results are τ -steps matched by zero ideal steps because α_S omits inactive prepared data and obs_{api} hides internal controller results. Stopping and restarting is matched by a τ self-loop on each committed-state machine by lemma 17.

Theorem 6 (Agent-API weak bisimulation after restart, detailed). *$\mathcal{B}$ is a divergence-insensitive weak bisimulation under obs_{api} .*

Proof. The least-generated rule sets in sections A-H and V and table III make the preceding cases exhaustive. For every step from either side, the corresponding case constructs a path with observation $\text{obs}_{\text{api}}(\ell)$ and a successor related by the witness in that matrix. The construction is symmetric because the ideal and runtime automata accept the same protected calls, while runtime-only steps preserve their common visible state. Arbitrary repetition of silent preload, internal NoCommit, Checkpoint, or restart steps is ignored, which is precisely divergence-insensitive weak matching. $\square$

M. Executable-Artifact Evaluation

The performance script runs every sample in a fresh CPython 3.12 process and reports the median of five repetitions on a 16-vCPU AMD EPYC 7R12. The 19 paper-specific tests exercise indexed pomset resolution, the greatest fixed point, and finite fixtures for all six constructors. The executable model covers the checker core and all six edit fixtures, while the theorem development establishes immutable edit rules, authority-approved outcome removal, and the complete domain-wide rule-version protocol.

AI USE ACKNOWLEDGMENT

OpenAI Codex was used throughout the manuscript for initial prose drafting and revision, and in the executable artifact for code scaffolding and test generation. It also served as an interactive aid for literature discovery, counterexample search, and the discussion and refinement of the formal theory, including definitions, assumptions, theorem statements, and proof structure. The authors chose and directed the research, made all final theoretical and methodological decisions, independently verified the cited sources, theorem statements, proofs, code, and experimental results, and remain responsible for every claim, proof, result, and citation.

REFERENCES

- [1] OpenAI, “Codex App Server,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://learn.chatgpt.com/docs/app-server.md>
- [2] Anthropic, “Manage sessions,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://code.claude.com/docs/en/sessions>
- [3] —, “Checkpointing,” Official documentation, 2026, accessed 2026-07-31. [Online]. Available: <https://code.claude.com/docs/en/checkpointing>
- [4] C. Wang and Y. Zheng, “Fork, explore, commit: Os primitives for agentic exploration,” 2026, agentic OS Workshop 2026. [Online]. Available: <https://arxiv.org/abs/2602.08199>
- [5] T. Wu, C. Chang, L. Cao, W. Gao, and W. Wang, “Crab: A semantics-aware checkpoint/restore runtime for agent sandboxes,” 2026. [Online]. Available: <https://arxiv.org/abs/2604.28138>
- [6] Y. Zhang, “Agent libos: A runtime substrate for capability-controlled self-evolving llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.03895>
- [7] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, “ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore,” 2026, coDAIM Workshop 2026. [Online]. Available: <https://arxiv.org/abs/2603.20625>
- [8] S. Yu, D. Chong, A. Nandi, D. Soylu, J. Sun, C. D. Manning, and W. Shi, “Shepherd: Enabling programmable meta-agents via reversible agentic execution traces,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.10913>
- [9] Y. Li, S. Ping, X. Chen, X. Qi, Z. Wang, Y. Luo, and X. Zhang, “AgentGit: A version control framework for reliable and scalable LLM-powered multi-agent systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.00628>
- [10] LangChain AI, “Use time-travel,” 2026, accessed 2026-08-03. [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/use-time-travel>
- [11] S. G. Patil, T. Zhang, V. Fang, N. C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, “GoEX: Perspectives and designs towards a runtime for autonomous LLM applications,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.06921>
- [12] E. Y. Chang and L. Geng, “SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning,” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4874–4886, 2025. [Online]. Available: <https://www.vldb.org/pvldb/vol18/p4874-chang.pdf>
- [13] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindshaedler, “Atomix: Timely, transactional tool use for reliable agentic workflows,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.14849>
- [14] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, “Cordon: Semantic transactions for tool-using llm agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.17573>
- [15] K. Yang, P. Li, Z. Wu, K. Xu, H. Huang, and X. Huang, “DART: Semantic recoverability for structured tool agents,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.23311>
- [16] M. H. de Queiroz, J. E. R. Cury, and W. M. Wonham, “Multitasking supervisory control of discrete-event systems,” *Discrete Event Dynamic Systems*, vol. 15, no. 4, pp. 375–395, 2005.
- [17] R. Malik and R. Leduc, “Generalised nonblocking,” in *Proceedings of the 9th International Workshop on Discrete Event Systems*, 2008, pp. 340–345.
- [18] B. Finkbeiner, F. Klein, and N. Metzger, “Live synthesis,” *Innovations in Systems and Software Engineering*, vol. 18, no. 3, pp. 443–454, 2022. [Online]. Available: <https://doi.org/10.1007/s11334-022-00447-5>
- [19] L. Aceto, I. Cassar, A. Francalanza, and A. Ingólfssdóttir, “On runtime enforcement via suppressions,” in *29th International Conference on Concurrency Theory (CONCUR 2018)*, ser. Leibniz International Proceedings in Informatics, vol. 118. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2018, pp. 34:1–34:17.
- [20] J. A. Brzozowski, “Derivatives of regular expressions,” *Journal of the ACM*, vol. 11, no. 4, pp. 481–494, 1964.
- [21] R. E. Strom and S. Yemini, “Optimistic recovery in distributed systems,” *ACM Transactions on Computer Systems*, vol. 3, no. 3, pp. 204–226, 1985.
- [22] E. Y. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, “A survey of rollback-recovery protocols in message-passing systems,” *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002.
- [23] W. M. P. van der Aalst and T. Basten, “Inheritance of workflows: An approach to tackling problems related to change,” *Theoretical Computer Science*, vol. 270, no. 1–2, pp. 125–203, 2002.
- [24] L. Nahabedian, V. A. Braberman, N. D’Ippolito, S. Honiden, J. Kramer, K. Tei, and S. Uchitel, “Dynamic update of discrete event controllers,” *IEEE Transactions on Software Engineering*, vol. 46, no. 11, pp. 1220–1240, 2020.
- [25] G. Amram, S. Maoz, I. Segall, and M. Yossef, “Dynamic update for synthesized GR(1) controllers,” in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 2022, pp. 786–797.
- [26] Q. Burke, A. Vahldiek-Oberwagner, M. Swift, and P. McDaniel, “It’s a feature, not a bug: Secure and auditable state rollback for confidential cloud applications,” 2025, accepted at the 2026 IEEE Symposium on Security and Privacy. [Online]. Available: <https://arxiv.org/abs/2511.13641>